Machine Learning for Mechanical Engineering

Final exam study sheet — ETH Zürich, 151-3231-00L, HS2026 (Prof. M. Fuge)

Built from the course textbook (IDEALLab/ML4ME_Textbook), PS1–PS2, the exercise solutions, the Moodle syllabus and the HS2025 review problems.

Contents

1 Exam format and how to use this sheet **4**

2 Supervised linear models, loss functions and penalties **4**

2.1 What "linear" means . . . 4
2.2 Least squares and ridge in closed form . . . 5
2.3 Regularization and L_p norms . . . 5
2.4 Loss functions for regression . . . 6
2.5 Loss functions for classification . . . 6
2.6 How to choose (summary) . . . 7

3 Evaluating models: cross-validation and hyperparameter search **7**

3.1 Why cross-validate . . . 7
3.2 Bias, variance and the curves you will be shown . . . 8
3.3 Hyperparameter search . . . 8
3.4 The curse of dimensionality . . . 9

4 Gradient descent and stochastic gradient descent **9**

4.1 The algorithm . . . 9
4.2 Gradients you must be able to write down . . . 10
4.3 How large can the step size be? (the most important idea) . . . 10
4.4 Stochastic and mini-batch gradient descent . . . 11
4.5 Momentum . . . 11
4.6 Adaptive optimizers and L-BFGS . . . 11

5 Unsupervised linear models and bilinear models **12**

5.1 One framework: $\mathbf{X} \approx \mathbf{ZW}$ . . . 12
5.2 PCA . . . 12
5.3 Comparison of the four decompositions . . . 13
5.4 Bilinear models: recommender with SGD (PS1 part 3) . . . 13

6 Automatic differentiation **14**

6.1 Four ways to get a derivative . . . 14
6.2 Forward mode (tangent propagation) . . . 14
6.3 Reverse mode (adjoints, backpropagation) . . . 14
6.4 Rules for elementary operations . . . 15
6.5 Finite differences . . . 15
6.6 Pitfalls (favourite conceptual questions) . . . 16

7 Neural networks, autoencoders and least volume **16**

7.1 Feed-forward networks (MLPs) . . . 16

7.2 Autoencoders . . . 17
7.3 Least volume (LV) regularization . . . 17

8 Generative models **18**
8.1 Generative adversarial networks (GANs) . . . 18
8.2 GAN training pitfalls and diagnostics . . . 18
8.3 Optimal transport generators . . . 19
8.4 Variational autoencoders (VAEs) . . . 19
8.5 Normalizing flows . . . 20
8.6 Continuous normalizing flows (neural ODEs) . . . 20
8.7 Score matching and Langevin sampling . . . 20
8.8 Diffusion models . . . 21
8.9 Flow matching . . . 21
8.10 Latent generative models . . . 22
8.11 Which model when . . . 22

9 Transformers and models for sets **22**
9.1 Self-attention . . . 22
9.2 MLP vs. PointNet vs. set transformer . . . 23

10 Measuring distances between distributions **23**
10.1 Closed forms for Gaussians . . . 23
10.2 Properties . . . 24

11 Inference: Bayesian regression, MCMC and variational inference **24**
11.1 Bayesian linear regression (closed form) . . . 24
11.2 Why we need approximations . . . 25
11.3 MCMC: Metropolis–Hastings . . . 25
11.4 Variational inference (VI) . . . 25
11.5 Probabilistic programming (Pyro) . . . 26

12 Reinforcement learning **26**
12.1 Setting . . . 26
12.2 The four methods . . . 26

13 Active learning and semi-supervised learning (small data) **27**
13.1 Active learning strategies . . . 27
13.2 Gaussian processes (uncertainty sampling, Bayesian optimization) . . . 27
13.3 Semi-supervised learning: label propagation and spreading . . . 28

14 Code reference for the JupyterHub part **28**
14.1 scikit-learn: linear models, CV, search . . . 28
14.2 PCA and decompositions . . . 28
14.3 PyTorch autograd, Jacobians, finite-difference checks . . . 29
14.4 Optimizing with autograd; constrained optimization with SciPy . . . 29
14.5 GAN training loop (correct version) and bugs to look for . . . 29
14.6 VAE, score matching, diffusion, flow matching . . . 30
14.7 Metropolis–Hastings . . . 30

15 Worked exam-style answers **30**

15.1 HS2025 review problem 3: model diagnostics (3 points) 30

15.2 Review problem 2: matching noise levels in denoising score matching 31

15.3 Review problem 1: gradient descent step sizes 31

15.4 Quick answers to likely conceptual questions 32

15.5 Plot-reading checklist 32

1 Exam format and how to use this sheet

Format. The final is a *digital session exam* in the ETH exam rooms. It has classic Moodle questions (multiple choice, short answers, matching plots) plus **JupyterHub code notebooks**. Problem sets you uploaded by the Moodle deadline may be available to you during the exam (PS1 was due 18.10, **PS2 is due 27.11.2026**). No cheatsheet is allowed in the exam, so this sheet is for studying and for solving practice problems.

What the questions look like. The HS2025 review problems had three tasks:

1. *Gradient descent*: given f and f' plots, say where GD converges (or diverges) for four step sizes → §4.
2. *Score matching*: match three noise levels σ to plots of the learned score field → §8.7.
3. *Model diagnostics*: read GAN code and its training plots, name the buggy line, explain why, and name the plot that shows it → §8.2.

So the exam rewards **reading plots and code and explaining mechanisms** more than long derivations. Each section below therefore has: the key formulas, the intuition behind them, typical plot signatures, and exam tips.

Dates	Lecture (Moodle syllabus)	Section
17.09–28.09	Review of linear models, loss and penalty functions, cross-validation	§2, §3
01.10	(Stochastic) gradient descent	§4
05.10–08.10	Unsupervised linear models and bi-linear models	§5
12.10–15.10	Automatic differentiation	§6
22.10–26.10	Neural networks and regularization, autoencoders	§7
29.10–02.11	GANs, debugging generative models (GAN pitfalls)	§8
05.11–12.11	Score matching, diffusion models, flow matching	§8
16.11–19.11	Transformers	§9
23.11	Latent generative models and review of generative models	§8
26.11	Measuring distances between distributions	§10
30.11–03.12	Inference (MCMC, variational inference), probabilistic programming	§11
07.12–10.12	Reinforcement learning and deep RL	§12
14.12	Active learning and semi-supervised learning	§13

The textbook also has chapters on optimal transport, VAEs and normalizing flows that do not appear as separate lectures. They are included (§8.3–§8.6) because the score-matching, diffusion and latent-model lectures build on them.

2 Supervised linear models, loss functions and penalties

2.1 What "linear" means

A linear model is linear *in its parameters* $\mathbf{w}$, not necessarily in the input x:

$$\hat{y} = \mathbf{w}^\top \boldsymbol{\phi}(x) = w_0\phi_0(x) + w_1\phi_1(x) + \cdots + w_p\phi_p(x).$$

The features $\phi_j(x)$ may be any *fixed* nonlinear functions, e.g. $\boldsymbol{\phi}(x) = [1, x, x^2, x^3]$ (polynomial regression) or $[1, t, \sin(\omega t)]$ with ω chosen in advance (PS1 CO_2 model). Because $\hat{y}$ is linear in $\mathbf{w}$, the squared-error problem is convex with a closed-form solution.

The "is it linear?" test

Freeze the input. If the prediction is then a weighted sum of *known* numbers with the unknown parameters as the weights, the model is linear. If a parameter sits *inside* a nonlinear function (an exponent, a sine, a denominator), it is not, unless a transformation of y or x pulls it out.

Model	Verdict	Reason / transformation
$y = w_0 + w_1 x + w_2 x^2$	linear	features $[1, x, x^2]$
$y = a e^{bx}$	linear after log	$\log y = \log a + b\,x$: features $[1, x]$, weights $[\log a, b]$
$y = w_0 + w_1 \log x + w_2 \sqrt{x}$	linear	features $[1, \log x, \sqrt{x}]$ (nonlinear in x is fine)
$y = w_1 \sin(w_2 x)$	not linear	w_2 is inside the sine
$y = a x^b$	linear after log–log	$\log y = \log a + b \log x$ (how S–N / Basquin fatigue curves are fit)
$y = w_0 + \dfrac{w_1}{w_2 + x}$	not linear	w_2 in the denominator; grid-search w_2, solve a linear problem for each
$y = \dfrac{1}{w_0 + w_1 x}$	linear after reciprocal	$1/y = w_0 + w_1 x$

Exam tip

Transforming y also transforms the noise. Additive Gaussian noise on y becomes multiplicative and skewed on $\log y$, and $1/y$ explodes for $y \approx 0$. Least squares on the transformed problem therefore weights the data differently. Always look at the residuals afterwards.

2.2 Least squares and ridge in closed form

Stack the features into the design matrix $\mathbf{X} \in \mathbb{R}^{N \times p}$ (row i is $\boldsymbol{\phi}(x_i)^\top$).

Key formulas

$$\text{OLS:} \quad \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \quad \Rightarrow \quad \mathbf{X}^\top \mathbf{X}\,\hat{\mathbf{w}} = \mathbf{X}^\top \mathbf{y} \quad \text{(normal equations)}, \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\text{Ridge:} \quad \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_2^2 \quad \Rightarrow \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \alpha I)^{-1} \mathbf{X}^\top \mathbf{y}$$

Derivation

$\nabla_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$ and $\nabla_{\mathbf{w}}\, \alpha \|\mathbf{w}\|^2 = 2\alpha\mathbf{w}$. Set the sum to zero: $(\mathbf{X}^\top \mathbf{X} + \alpha I)\mathbf{w} = \mathbf{X}^\top \mathbf{y}$. Adding αI raises every eigenvalue of $\mathbf{X}^\top \mathbf{X}$ by α, so the matrix becomes invertible and well-conditioned even when features are nearly collinear (e.g. x^{14} and x^{15} on $[0, 1]$).

2.3 Regularization and L_p norms

General cost: $\text{Cost}(\mathbf{w}) = \text{Loss}(\mathbf{w}, D) + \alpha\, \Omega(\mathbf{w})$, where α is the "price" of model complexity.

$$\|\mathbf{x}\|_p = \Big(\sum_{i=1}^{n} |x_i|^p \Big)^{1/p}, \qquad \|\mathbf{x}\|_2 = \sqrt{x_1^2 + \cdots + x_n^2}, \quad \|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|.$$

Penalty $\Omega(\mathbf{w})$	Effect on weights	Gradient	Use when
L_2: $\|\mathbf{w}\|_2^2$ (ridge, Tikhonov, Gaussian prior)	Shrinks all weights smoothly towards 0, rarely exactly 0	$2\alpha\mathbf{w}$: shrinks as $w \to 0$	Correlated / redundant inputs, ill-conditioned problems, stability wanted
L_1: $\|\mathbf{w}\|_1$ (LASSO, Laplace prior)	Sets many weights *exactly* to 0 (sparsity, feature selection)	$\alpha\,\text{sign}(\mathbf{w})$: constant size; at 0 any value in $[-\alpha, \alpha]$	Many candidate features, want the few that matter, $n \ll p$, interpretability
L_0 (count of nonzeros)	Ideal sparsity	not differentiable	impractical to optimize
$p < 1$ (e.g. 0.5)	Non-convex, jumps to zero at very small α		rarely used
L_∞ ($p \to \infty$)	Penalizes only the largest weight: weights equalize first, then shrink together		rarely sensible as a penalty
Elastic net $\alpha\|\mathbf{w}\|_2 + \beta\|\mathbf{w}\|_1$	Sparse, but shares weight among correlated features		LASSO picks one of a correlated group arbitrarily

Why L_1 gives exact zeros and L_2 does not

Gradient view: the L_2 pull $2\alpha w$ vanishes as $w \to 0$, so a weight only gets small. The L_1 pull $\alpha\,\mathrm{sign}(w)$ stays at full strength right down to zero; a weight stays at exactly 0 whenever the data gradient there is smaller than α in magnitude.
Geometry view: the optimum is where the loss contours first touch the penalty contour $\{\Omega(\mathbf{w}) = c\}$. The L_1 ball is a diamond with corners on the axes, so the touching point is usually a corner, i.e. some $w_j = 0$. The L_2 ball is round, so it touches at a generic point.

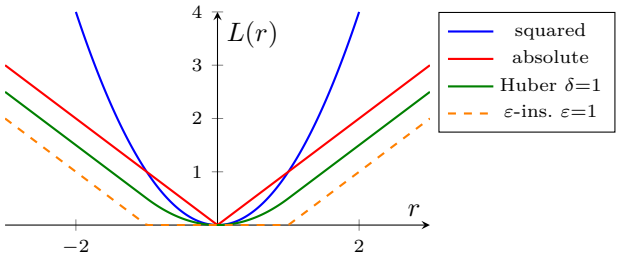
Recognizing penalty plots

Contour shapes: **star** (inward-curving) = $p < 1$; **diamond** = L_1; **circle** = L_2; **rounded square** = L_∞. Regularization paths as α grows: L_1 runs straight to an axis, then slides along it to the origin (one coefficient hits 0 at a moderate α and stays there). L_2 bends smoothly towards the origin, nothing hits zero early. $p = 0.5$ jumps to the axis abruptly at small α. L_∞ shrinks the larger coefficient first until both have equal magnitude, then shrinks them together (the smaller one can even grow temporarily).

2.4 Loss functions for regression

Let $r = y - \hat{y}$ be the residual.

Loss	$L(r)$	Behaviour and when to use
Squared (L_2, MSE)	r^2	Default choice. Under the Gauss–Markov assumptions (uncorrelated, equal-variance, zero-mean errors) OLS is the Best Linear Unbiased Estimator (BLUE). Large errors are squared, so *outliers dominate* the fit.
Absolute (L_1)	$\lvert r\rvert$	Robust: every point pulls with the same force. Fits the conditional median.
Huber (δ)	$\begin{cases} \frac{1}{2}r^2 & \lvert r\rvert \le \delta \\ \delta(\lvert r\rvert - \frac{1}{2}\delta) & \lvert r\rvert > \delta \end{cases}$	Quadratic near 0 (smooth, efficient), linear in the tails (robust). $\frac{dL}{dr} = r$ for $\lvert r\rvert \le \delta$, else $\delta\,\mathrm{sign}(r)$: an outlier's pull is capped at δ.
ε-insensitive	$\max(0, \lvert r\rvert - \varepsilon)$	Zero loss inside a band of width ε, so only points outside the band (support vectors) shape the fit. Smaller ε means more support vectors.



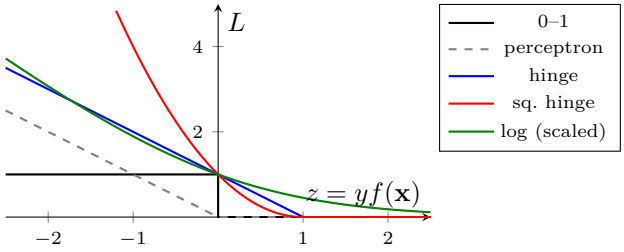
"Which loss made this fit?"

With a cluster of outliers, the **squared loss** is the odd one out: the line is dragged towards the outliers, so its residual histogram is wide and shifted away from zero for the good points. Huber and ε-insensitive fits look *practically identical*, because both grow linearly for large errors, so outliers pull with the same constant force. They only differ for small errors (few data, no outliers, skewed noise).

2.5 Loss functions for classification

Labels $y \in \{-1, +1\}$, decision value $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, **margin** $z = y\,f(\mathbf{x})$. Then $z > 0$ means correctly classified, and larger z means more confident.

Loss	$L(z)$	Notes
Zero–one	$\mathbb{1}[z \le 0]$	What we actually care about, but not differentiable, so not usable for training
Perceptron	$\max(0, -z)$	Zero for every correctly classified point, so no unique solution (any separating line is optimal)
Hinge (linear SVM)	$\max(0, 1 - z)$	Also penalizes correct points with $0 < z < 1$ (inside the margin). Kink at $z = 1$. Approximately maximizes the margin
Squared hinge	$\max(0, 1 - z)^2$	Smooth; punishes large violations harder
Log loss (logistic regression)	$\log(1 + e^{-z})$	Gives calibrated probabilities $P(y = 1 \mid \mathbf{x}) = \sigma(f) = \frac{1}{1+e^{-f}}$. Never exactly zero
Modified Huber	$\begin{cases} \max(0, 1 - z)^2 & z \ge -1 \\ -4z & z < -1 \end{cases}$	Squared hinge that becomes linear for badly misclassified points (robust)



Reading a hinge-loss plot

The solid line is $f(\mathbf{x}) = 0$, the dashed lines are $f(\mathbf{x}) = \pm 1$. A point beyond the dashed line on its *own* side: $z > 1$, loss 0. Exactly on its own dashed line: $z = 1$ (the kink). Between the solid line and its own dashed line: $0 < z < 1$, small positive loss. On the solid line: $z = 0$, loss 1. On the wrong side: $z < 0$, loss $1 - z > 1$. The perceptron loss is 0 for every correct point; the hinge loss is nonzero only for points within ± 1 of the boundary or misclassified.

2.6 How to choose (summary)

- **Regression:** no prior knowledge ⇒ squared loss. Outliers or spurious large readings ⇒ Huber or absolute. Want the fit to depend on only a few points ⇒ ε-insensitive (SVR).
- **Classification:** need probabilities ("95 % sure the part failed") ⇒ log loss. Want maximum margin ⇒ hinge. Zero–one and perceptron are not used in practice.
- **Penalty:** stability with redundant inputs ⇒ L_2. Few important features out of many ⇒ L_1. Both ⇒ elastic net.

Exercise: thrown object tracked by a flaky lidar

Free flight $h(t) = h_0 + v_0 t - \frac{1}{2} g t^2$ is linear in $(h_0, v_0, -g/2)$ with features $[1, t, t^2]$. About one reading in eight drops out to exactly 0. Those are huge, one-sided outliers, so squared loss lets them dominate: pulling the parabola down mainly reduces its curvature, so the estimate of g suffers most, while h_0 (anchored by early readings) moves least. Use **Huber** (`HuberRegressor`). With plain `SGDRegressor`, standardize t and t^2 first, because they live on very different scales.

3 Evaluating models: cross-validation and hyperparameter search

3.1 Why cross-validate

Training error is an optimistic estimate of performance, because the model has seen those points. We want the *out-of-sample* (test) error. Cross-validation (CV) repeatedly holds out part of the data, fits on the rest and scores on the held-out part, then averages:

$$\widehat{\mathrm{Err}}_{CV} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|D_k|} \sum_{i \in D_k} L\big(y_i, \hat{f}^{(-k)}(x_i)\big),$$

where $\hat{f}^{(-k)}$ is trained without fold D_k. Leave-one-out (LOO) is $K = N$.

The golden rule

The split must mirror how the model will be used. Ask: *"In deployment, what will be new to the model?"* Whatever that is (new time steps, new parts, new plants) must be absent from the training folds. Any preprocessing that learns from data (scaler, PCA, feature selection) must also be fit *inside* each training fold, otherwise it has already seen the test rows (leakage). Use a `Pipeline`.

Situation	Split	What leaks with a plain shuffled split
Independent rows (40 separately cast and tested specimens)	Shuffled K-fold (LOO fine for small N)	Nothing: rows are i.i.d.
Rare class (2 % defective welds; 5 faults in 1000 engines)	Stratified K-fold on the label	Some folds contain no positives, so precision/recall are undefined and scores are noisy
Many rows per unit, deployment on **new** units (blades, plants, patients)	Group K-fold on the unit (leave-one-group-out)	Rows of the same blade on both sides: the model memorizes unit-specific quirks
Forecasting (strain one hour ahead, CO_2 series)	Time-series split: train on the past, test on the future	Model trains on the future; optimistic error
Forecasting the future of the **same** units (remaining life of these 20 cells)	Time split within each unit	Each unit's future leaks into its past (grouping by unit would answer the wrong question)

3.2 Bias, variance and the curves you will be shown

$$\mathbb{E}\big[(y-\hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)]-f(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}[\hat{f}(x)]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}.$$

- **High bias (underfit):** model too simple or penalty too strong. Train and test error both high and close together. More data does not help.
- **High variance (overfit):** model too flexible or penalty too weak. Train error low, test error high; fitted curves change wildly between folds. More data or more regularization helps.

Validation curve (test error vs. penalty α or complexity)

Left end (tiny α, high degree): variance dominates, test error high. Right end (huge α): weights shrunk to ≈ 0, a flat line, bias dominates. The minimum in between is the chosen hyperparameter. A 15-degree polynomial on $[0,1]$ has nearly collinear columns, so the unregularized problem is close to singular and the weights explode. Even $\alpha = 10^{-9}$ dramatically stabilizes it (it lifts the near-zero eigenvalues), after which the curve stays flat for many decades of α.

Learning curve (error vs. training size N)

A simple model: train and test errors meet quickly at a high plateau (bias). A flexible model: big gap at small N that shrinks as N grows (variance). **PS1 (CO_2):** at $N = 50$–100 the linear + sine model wins (3 parameters capture the seasonal cycle; the quadratic term is barely determined over one year and extrapolates wildly, even though Model 4 has the lowest *training* error). At $N = 2000$ (≈40 years) the quadratic + sine model wins: CO_2 growth visibly accelerates, so the linear trend underfits (its training error rises too), while there is now enough data to pin down the t^2 coefficient.

3.3 Hyperparameter search

Grid search. Every combination on a grid. Number of fits = (#settings) × (#folds), plus one final refit. Example: $15 \times 10 \times 2 = 300$ settings × 20 splits = 6000 fits; at 30 s each that is 50 hours. With d hyperparameters and m values each, the cost is m^d.

Randomized search. Same budget, sample settings from distributions. Better coverage when only a few hyperparameters actually matter. Budget trade-off: more settings (coverage) vs. more folds (less noisy score per setting). A usual compromise is 12–20 settings × 3–5 folds.

Hyperparameter	Distribution	Why
Ridge α, learning rate	`loguniform(1e-8, 1e3)`	Effect is multiplicative over decades; every decade deserves equal draws. A *uniform* draw on $(10^{-8}, 10^{3})$ lands above 100 with probability 0.9!
Polynomial degree	`randint(1, 21)`	Small integer range (upper bound excluded)
Subsample fraction	`uniform(0.05, 0.9)` $\rightarrow [0.05, 0.95]$	Bounded, linear scale
Hidden units	$2^{\texttt{randint(3,11)}}$ (8 to 1024)	8→16 matters as much as 512→1024
Phase angle	`uniform(-π, 2π)`	Bounded and periodic, no scale preferred

Bayesian optimization. Fit a cheap probabilistic surrogate (usually a Gaussian process, §13) to the evaluations so far. It gives a mean $\mu(x)$ and uncertainty $\sigma(x)$ everywhere. Pick the next point by maximizing an *acquisition function* that trades off exploitation (high μ) against exploration (high σ):

Key formulas

$$\mathrm{UCB}(x) = \mu(x) + \kappa\,\sigma(x) \quad \text{(maximizing } f\text{)}, \qquad \mathrm{LCB}(x) = \mu(x) - \kappa\,\sigma(x) \quad \text{(minimizing } f\text{)}$$

- $\kappa = 0$: greedy, picks the highest mean. κ large (e.g. 10): picks the largest uncertainty, the mean hardly matters. $\kappa = 1.96$: the top of the 95 % band.
- A point next to existing observations has tiny σ, so its UCB barely exceeds its mean. A point in a wide gap with a moderate mean often wins.
- An already evaluated (noiseless) point has $\sigma = 0$, so UCB = its value; there is always an unobserved point with a higher bound. **BO never re-evaluates a noiseless observation.**
- Worth it when each evaluation is expensive (a simulation, a long training run).

3.4 The curse of dimensionality

- **A Gaussian is a donut.** In n dimensions the distance r of a standard normal sample from the mean has density $p(r) \propto r^{n-1}e^{-r^2/2}$ (chi distribution). Setting $\frac{d}{dr}\log p = \frac{n-1}{r} - r = 0$ gives the mode $r^* = \sqrt{n-1}$. In 10-D only $\approx 0.02\,\%$ of samples fall inside the unit ball and $\approx 5\,\%$ inside $r < 2$; most lie between $r \approx 2$ and 4. The density per unit volume is highest at the mean, but there is almost no volume there. A typical sample does not look like the most likely sample.
- **Distances concentrate.** The ratio nearest/farthest neighbour distance tends to 1 as d grows: everyone looks equally far away. This hurts k-NN and kernel/distance weighting (the ordering is unchanged, the contrast is lost).
- **Search.** A grid needs m^d points; random points are almost always far from any particular good region and most volume sits in the "corners". This motivates model-based search (BO).

4 Gradient descent and stochastic gradient descent

4.1 The algorithm

For a cost $J(\mathbf{w})$, e.g. $J(\mathbf{w}) = \frac{1}{N}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \alpha\,\Omega(\mathbf{w})$, take small steps against the gradient:

Key formulas

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\,\nabla_{\mathbf{w}} J(\mathbf{w}_t) \qquad (\alpha = \text{step size / learning rate}).$$

Gradient of the MSE for the slope

$$\frac{\partial J}{\partial w} = \frac{\partial}{\partial w}\frac{1}{N}\sum_i (y_i - w x_i)^2 = \frac{2}{N}\sum_i (y_i - w x_i)\cdot\frac{\partial}{\partial w}(y_i - w x_i) = -\frac{2}{N}\sum_i (y_i - w x_i)\,x_i.$$

Chain rule on a square: bring down the 2, multiply by the derivative of the inside. The gradient of a sum is the sum of the gradients.

4.2 Gradients you must be able to write down

Term	Gradient
MSE with intercept, $r_i = y_i - ax_i - b$	$\frac{\partial J}{\partial a} = -\frac{2}{N}\sum_i r_i x_i, \quad \frac{\partial J}{\partial b} = -\frac{2}{N}\sum_i r_i$ (the intercept is a feature that is always 1)
$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$	$-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$; setting it to 0 gives $\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$
$\mathbf{w}^\top A\mathbf{w}$	$(A + A^\top)\mathbf{w}$, which is $2A\mathbf{w}$ for symmetric A
$\|\mathbf{Y} - \mathbf{X}\mathbf{W}\|_F^2$	$-2\mathbf{X}^\top(\mathbf{Y} - \mathbf{X}\mathbf{W})$ (the vector case, column by column)
$\mathbf{u}^\top\mathbf{v}$ w.r.t. $\mathbf{v}$	$\mathbf{u}$ (the other vector acts as a constant)
L_2 penalty $\alpha\|\mathbf{w}\|_2^2$	$2\alpha\mathbf{w}$
L_1 penalty $\alpha\|\mathbf{w}\|_1$	α sign$(\mathbf{w})$ for $w_j \neq 0$; at $w_j = 0$ any subgradient in $[-\alpha, \alpha]$
Huber	r for $\|r\| \le \delta$, else δ sign(r) (times $\partial r/\partial \mathbf{w} = -\mathbf{x}$)
Log loss $\log(1 + e^{-z})$, $z = y\mathbf{w}^\top\mathbf{x}$	$\nabla_{\mathbf{w}} L = -\frac{e^{-z}}{1+e^{-z}}\, y\mathbf{x} = -\big(1 - \sigma(z)\big)\, y\,\mathbf{x}$

A confidently correct example ($z \gg 0$) contributes almost nothing to the log-loss gradient; a misclassified one pulls with weight close to 1. When checking an L_1 gradient with finite differences, stay away from $w = 0$: a difference that straddles zero averages the two one-sided slopes.

4.3 How large can the step size be? (the most important idea)

Near a minimum w^*, write the error $e_t = w_t - w^*$ and Taylor-expand the gradient: $J'(w) \approx J'(w^*) + J''(w^*)(w - w^*) = \lambda\, e$, where $\lambda = J''(w^*)$ is the **curvature**. Substituting into the update:

Key formulas

$$e_{t+1} = e_t - \alpha\lambda e_t = (1 - \alpha\lambda)\, e_t \quad \Longrightarrow \quad e_t = (1 - \alpha\lambda)^t e_0.$$

GD converges iff $|1 - \alpha\lambda| < 1$, i.e. $\boxed{0 < \alpha < 2/\lambda}$. In several dimensions, λ is the *largest* Hessian eigenvalue.

$\alpha\lambda$	factor $1 - \alpha\lambda$	behaviour
$\ll 1$	close to 1	monotone but very slow
$= 1$ ($\alpha = 1/\lambda$)	0	lands on the optimum in one step (quadratic J)
between 1 and 2	between -1 and 0	zig-zags around the minimum but converges
$= 2$	-1	bounces between two points forever
> 2	< -1	every step overshoots more: diverges

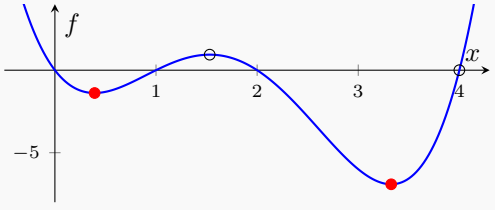
For the one-parameter MSE fit $J(w) = \frac{1}{N}\sum(y_i - wx_i)^2$, the curvature is $\lambda = 2\overline{x^2}$, so the best step is $\alpha = 1/(2\overline{x^2})$ and divergence starts at $\alpha = 1/\overline{x^2}$.

Intuition

Poorly scaled features make the curvature very different in different directions (an elongated valley). The step must be small enough for the steepest direction ($\alpha < 2/\lambda_{\max}$), so progress along the flat direction ($\lambda_{\min}$) is slow and the path zig-zags. Standardizing features (zero mean, unit variance) makes the contours rounder, so one α suits all directions.

HS2025 review problem 1 / textbook exercise: where does GD end up?

$f(x) = x^4 - 7x^3 + 14x^2 - 8x$, $f'(x) = 4x^3 - 21x^2 + 28x - 8$, start $x_0 = 4$, where $f(4) = 0$ and $f'(4) = 24$. Stationary points: local min $x \approx 0.393$ ($f'' \approx 13.4$), local max $x \approx 1.531$, global min $x \approx 3.326$ with $f'' \approx 21.1$ (so stable only for $\alpha < 2/21.1 \approx 0.095$).



Recipe: (1) one step $x_1 = x_0 - \alpha f'(x_0) = 4 - 24\alpha$; (2) which basin does x_1 land in (left or right of the hump at 1.531)? (3) is $\alpha f'' < 2$ at that basin's minimum?

α	x_1	outcome
A: 0.025	3.4	inside the right valley, $\alpha f'' = 0.53 < 2$: converges to the global minimum 3.326
B: 0.10	1.6	just right of the hump, creeps right into the right valley, but $\alpha f'' = 2.1 > 2$: bounces between ≈ 3.105 and 3.483 forever
C: 0.15	0.4	jumps over the hump, lands next to the left minimum 0.393 (a *local* minimum); oscillates $\approx 0.369 \leftrightarrow$ 0.419
D: 0.2	-0.8	steep left wall, $f'(-0.8) \approx -46$, next $x_2 \approx 8.4$, gradient even larger: diverges

The step size decides *which* valley you reach (where the early large steps land) and *whether* you settle (curvature at the bottom).

4.4 Stochastic and mini-batch gradient descent

SGD replaces the full gradient by the gradient of one sample (or a mini-batch of B samples), an unbiased but noisy estimate. One pass over the data is an *epoch*. With a constant α the iterates never settle; they jitter in a band around the optimum whose width scales with α. Mini-batches interpolate between SGD (noisy, cheap per step) and full GD (smooth, expensive per step).

Robbins–Monro conditions for exact convergence of SGD

$$\sum_{n=1}^{\infty} a_n = \infty \quad \text{(you can travel arbitrarily far)}, \qquad \sum_{n=1}^{\infty} a_n^2 < \infty \quad \text{(the noise averages out)}.$$

schedule	$\sum a_n = \infty$?	$\sum a_n^2 < \infty$?	verdict
a/n	yes (harmonic)	yes ($\pi^2/6$)	passes both (but often stalls in practice)
$a/\sqrt{n}$	yes	no (harmonic)	fails 2nd: keeps jittering in a small band (fine in practice)
constant a	yes	no	fails 2nd
a/n^2	no ($a\pi^2/6$)	yes	fails 1st: total travel is finite, may never arrive

scikit-learn's `SGDRegressor` with `learning_rate='invscaling'` uses $\eta_t = \texttt{eta0}/t^{\texttt{power_t}}$ (defaults `eta0`= 0.01, `power_t`= 0.25). Set `tol=None` to force exactly `max_iter` passes.

4.5 Momentum

Key formulas

$$\mathbf{v}_{t+1} = \beta\,\mathbf{v}_t + \nabla J(\mathbf{w}_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\,\mathbf{v}_{t+1}.$$

A "heavy ball" that keeps moving in the direction it was already going: it averages out zig-zag components and accelerates along consistent directions.

Stability with momentum

Near the optimum the gradient is λe_t. Using $\alpha v_t = e_{t-1} - e_t$:

$$e_{t+1} = (1 + \beta - \alpha\lambda)\,e_t - \beta\,e_{t-1}.$$

Try $e_t = r^t$: $r^2 - (1 + \beta - \alpha\lambda)\,r + \beta = 0$. Both roots satisfy $|r| < 1$ iff $0 < \alpha < \dfrac{2(1+\beta)}{\lambda}$. So momentum *raises* the usable step size (for $\beta = 0$ it reduces to $\alpha < 2/\lambda$). Large β means oscillations decay slowly ($|r| = \sqrt{\beta}$ when the roots are complex).

4.6 Adaptive optimizers and L-BFGS

Adam keeps running averages of the gradient g and of its square:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\,\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

with bias corrections $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$. Each coordinate gets its own step size. The first step moves every coordinate by $\approx \alpha$ against the sign of its gradient, *whatever the gradient's size*. Steps do not shrink just because the gradient is small, so Adam can cross shallow ridges or keep drifting downhill (on the McCormick function it slides valley to valley). **AdamW** adds decoupled weight decay (pulls weights towards 0 each step).

L-BFGS is quasi-Newton: it builds an approximation of the inverse Hessian from recent gradient differences and steps to the minimum of the local quadratic model. It is very fast on smooth, deterministic, low-dimensional problems (e.g. fitting damping γ and stiffness k to an oscillation record: 5 calls vs. Adam still drifting after 60 steps). It converges to the *nearest* minimum and has no mechanism to leave it. In PyTorch it needs a `closure` that recomputes the loss.

5 Unsupervised linear models and bilinear models

5.1 One framework: $\mathbf{X} \approx \mathbf{ZW}$

Data $\mathbf{X} \in \mathbb{R}^{n\times d}$ (n samples, e.g. airfoils with $d = 384$ coordinates) is approximated by *codes* (latent coordinates) $\mathbf{Z} \in \mathbb{R}^{n\times k}$ times *components* (basis shapes, atoms) $\mathbf{W} \in \mathbb{R}^{k\times d}$, with $k \ll d$. Each sample is a weighted sum of k basis shapes: $\mathbf{x}_i \approx \sum_j z_{ij}\,\mathbf{w}_j$. The methods differ only in the constraints and penalties.

Key formulas

PCA:	$\min_{\mathbf{W}} \ \lVert\mathbf{X} - \mathbf{X}\mathbf{W}^\top\mathbf{W}\rVert_F^2$ s.t. $\mathbf{W}\mathbf{W}^\top = I$	($\mathbf{Z} = \mathbf{X}\mathbf{W}^\top$, $\mathbf{X}$ centered)
Sparse PCA:	$\min_{\mathbf{Z},\mathbf{W}} \ \lVert\mathbf{X} - \mathbf{Z}\mathbf{W}\rVert_F^2 + \alpha\lVert\mathbf{W}\rVert_1$	(sparse components)
Dictionary learning:	$\min_{\mathbf{Z},\mathbf{W}} \ \lVert\mathbf{X} - \mathbf{Z}\mathbf{W}\rVert_F^2 + \alpha\lVert\mathbf{Z}\rVert_1$ s.t. $\lVert\mathbf{w}_j\rVert_2 \le 1$	(sparse codes)
NMF:	$\min_{\mathbf{Z}\ge 0,\,\mathbf{W}\ge 0} \ \lVert\mathbf{X} - \mathbf{Z}\mathbf{W}\rVert_F^2$ (+ small L_1/L_2 terms)	(non-negative parts)

5.2 PCA

PCA: maximum projected variance gives the covariance eigenvectors

Centered data, covariance $\Sigma = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$. The variance of the projections onto a unit vector $\mathbf{w}$ is $\mathbf{w}^\top\Sigma\mathbf{w}$. Maximize it subject to $\lVert\mathbf{w}\rVert = 1$ with a Lagrange multiplier:

$$\mathcal{L} = \mathbf{w}^\top\Sigma\mathbf{w} - \lambda(\mathbf{w}^\top\mathbf{w} - 1) \ \Rightarrow\ \Sigma\mathbf{w} = \lambda\mathbf{w}, \qquad \mathbf{w}^\top\Sigma\mathbf{w} = \lambda.$$

So the best direction is the eigenvector with the largest eigenvalue, and the variance captured is that eigenvalue. The next component is the next eigenvector (orthogonal to the first), and so on. Equivalently, PCA minimizes the squared reconstruction error.

PCA via SVD (how to do it by hand or in numpy)

Center: $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{x}}$. SVD: $\mathbf{X}_c = USV^\top$ (singular values $s_1 \ge s_2 \ge \dots$).

$$\text{components} = \text{rows of } V^\top, \qquad \mathbf{Z} = \mathbf{X}_c V = US, \qquad \lambda_j = \frac{s_j^2}{n}, \qquad \text{explained variance ratio}_j = \frac{s_j^2}{\sum_i s_i^2}.$$

Reconstruction with k components: $\hat{\mathbf{x}} = \bar{\mathbf{x}} + \sum_{j=1}^{k} z_j\,\mathbf{v}_j$ with $z_j = \mathbf{v}_j^\top(\mathbf{x} - \bar{\mathbf{x}})$.

- **Sign ambiguity.** If $\mathbf{v}$ is a singular vector so is $-\mathbf{v}$; flipping it flips the matching column of $\mathbf{Z}$, so $\mathbf{ZW}$ is unchanged. sklearn and numpy may disagree on signs. A component's sign means nothing on its own.
- **Nested and ordered.** The first k components do not change if you ask for more or fewer (they are the top eigenvectors). Not true for Sparse PCA, dictionary learning or NMF, which fit all components jointly (NMF components are not even ordered).
- **Uncorrelated codes.** $\mathrm{Cov}(\mathbf{Z}) = \mathbf{W}\Sigma\mathbf{W}^\top = \mathrm{diag}(\lambda_1, \dots, \lambda_k)$.
- **Scree plot and intrinsic dimension.** Plot λ_j (or explained variance ratio, or its cumulative sum) against j. Pick k at the elbow or where the cumulative ratio reaches a target (e.g. 95 %). In code see §14.

PS1 part 2

Toy 7-D data (hb.csv): two components explain $\approx 100\,\%$ of the variance, so the intrinsic dimension is 2. Projected to 2-D, the points form a regular grid tracing a 2-D silhouette image: the data is a flat picture linearly embedded in 7-D (possibly rotated or mirrored, because PC signs and orientation are arbitrary).
Topology-optimization images (94 178 pixels): PC1 explains only $\approx 15\,\%$, the scree curve decays gradually with no elbow, and 169 components are needed for $95\,\%$. Pixels are highly correlated (so far fewer than 94 178 dimensions), but the map from load parameters to optimal geometry is *nonlinear*, so a linear method needs many components. The top PCs are interpretable material-placement patterns (where material is added or removed relative to the mean bridge).

Denoising a scan by projection

Noisy scan $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\varepsilon}$, projection P_k onto the first k PCs, cleaned shape $\bar{\mathbf{x}} + P_k(\tilde{\mathbf{x}} - \bar{\mathbf{x}})$. Its error against the true shape splits into two orthogonal parts:

$$\underbrace{-(I - P_k)(\mathbf{x} - \bar{\mathbf{x}})}_{\text{truncation: falls with } k} + \underbrace{P_k \boldsymbol{\varepsilon}}_{\text{passed noise: } \mathbb{E}\|\cdot\|^2 = k\sigma^2} .$$

So the error vs. the *true* shape is U-shaped in k (RMS noise per coordinate $\sigma\sqrt{k/d}$), while the error vs. the *scan* always decreases. More noise means a smaller optimal k (noise 0.001: $k = 33$; 0.003: $k = 18$; 0.01: $k = 9$). Choose k without the truth by cross-validating hidden points within each scan.

In-painting (filling a gap): fit the latent coordinates by least squares to the observed points only, then predict the missing points from $\bar{\mathbf{x}} + \sum_j z_j \mathbf{v}_j$. It works when the observed and missing parts are strongly correlated (filling 51 missing points of an airfoil); it fails when they are not (the upper surface does not determine the lower surface). Least squares returns one answer, not a distribution, so tolerance decisions need a probabilistic model.

Uncorrelated $\neq$ independent

Sampling PCA codes as independent Gaussians reproduces only the mean and covariance of the data. Real design spaces have edges and constraints (non-negative camber, positive thickness, no self-intersection) that a linear-Gaussian model cannot represent. Sampling Sparse-PCA codes independently gives wavy, unrealistic airfoils because its codes *are* correlated. This is the motivation for nonlinear generative models (§8).

5.3 Comparison of the four decompositions

Property	PCA	Sparse PCA	Dict. learning	NMF
Components mutually orthogonal	**yes** ($\mathbf{W}\mathbf{W}^\top = I$)	no	no (atoms often nearly parallel)	no
Many exact zeros in $\mathbf{W}$ (localized parts)	no	**yes** (L_1 on $\mathbf{W}$)	no	a few (clipping)
Many exact zeros in $\mathbf{Z}$ (each sample uses few atoms)	no	no	**yes** (L_1 on $\mathbf{Z}$)	a few
No negative entries in $\mathbf{W}, \mathbf{Z}$ (additive parts)	no	no	no	**yes**
First component unchanged when k changes	**yes**	no	no	no
Codes uncorrelated	**yes**	no	no	no

Two non-negative vectors can only be orthogonal if they are never nonzero at the same coordinate, which is why NMF components overlap strongly.

5.4 Bilinear models: recommender with SGD (PS1 part 3)

Each user u and movie m gets a latent vector ($\mathbf{v}_u, \mathbf{v}_m \in \mathbb{R}^K$, $K = 2$ in class) and a scalar bias; μ is a global offset:

Key formulas

$$\hat{y}_{um} = \mathbf{v}_u^\top \mathbf{v}_m + b_u + b_m + \mu, \qquad C = \sum_{(u,m)\in R} \tfrac{1}{2} e_{um}^2 + \tfrac{\lambda}{2}\Big(\sum_u \left[\|\mathbf{v}_u\|^2 + b_u^2\right] + \sum_m \left[\|\mathbf{v}_m\|^2 + b_m^2\right]\Big),$$

with the error $e_{um} = \hat{y}_{um} - y_{um}$ and R the set of observed ratings.

It is called bilinear because it is linear in $\mathbf{v}_u$ for fixed $\mathbf{v}_m$ and vice versa. Only ratings that involve a given user (movie) depend on that user's (movie's) parameters, so all other terms have zero derivative:

Gradients

$$\frac{\partial C}{\partial \mathbf{v}_u} = \sum_{m \in R(u)} e_{um}\,\mathbf{v}_m + \lambda \mathbf{v}_u, \qquad \frac{\partial C}{\partial \mathbf{v}_m} = \sum_{u \in R(m)} e_{um}\,\mathbf{v}_u + \lambda \mathbf{v}_m,$$

$$\frac{\partial C}{\partial b_u} = \sum_{m \in R(u)} e_{um} + \lambda b_u, \qquad \frac{\partial C}{\partial b_m} = \sum_{u \in R(m)} e_{um} + \lambda b_m, \qquad \frac{\partial C}{\partial \mu} = \sum_{(u,m) \in R} e_{um}.$$

SGD update for one observed rating (u, m): compute $e = \hat{y}_{um} - y_{um}$, then update *simultaneously* (use the old $\mathbf{v}_u$ in the $\mathbf{v}_m$ update):

$$\mathbf{v}_u \leftarrow \mathbf{v}_u - \alpha(e\,\mathbf{v}_m + \lambda \mathbf{v}_u), \quad \mathbf{v}_m \leftarrow \mathbf{v}_m - \alpha(e\,\mathbf{v}_u + \lambda \mathbf{v}_m), \quad b_u \leftarrow b_u - \alpha(e + \lambda b_u), \quad b_m \leftarrow b_m - \alpha(e + \lambda b_m), \quad \mu \leftarrow \mu - \alpha e.$$

Missing ratings are predicted with $\hat{y}_{um}$. In the PS, μ starts at 2.5 and α between about 0.01 and 0.7 converges within 1000 epochs.

6 Automatic differentiation

6.1 Four ways to get a derivative

Method	Benefits	Drawbacks
Analytical (by hand)	exact, valid everywhere	tedious, error-prone
Symbolic (SymPy)	exact, a formula valid everywhere	slow, *expression swell*
Numerical (finite differences)	trivial to implement, works on any black box	approximate, sensitive to h, round-off, one point at a time, one evaluation per input
Automatic (PyTorch autograd)	exact to machine precision, efficient, no swell	one point at a time, implementation overhead, differentiates only the code path actually executed

Idea of AD: any program is a composition of elementary operations $(+, \times, \sin, \exp, \dots)$ whose derivatives are known. Write the computation as a graph of intermediate variables v_i and apply the chain rule node by node, with numbers (not formulas).

6.2 Forward mode (tangent propagation)

Carry, next to every value v, its *tangent* $\dot{v} = \partial v / \partial x$ along a chosen input direction (the *seed* $\dot{\mathbf{x}}$). One pass gives the directional derivative $J\dot{\mathbf{x}}$, i.e. **one column of the Jacobian** (seed = unit vector $\mathbf{e}_i$), plus the derivative of *every intermediate node* w.r.t. that input for free.

6.3 Reverse mode (adjoints, backpropagation)

First a forward pass stores all values. Then propagate *adjoints* $\bar{v} = \partial f / \partial v$ backwards from the output (seed $\bar{f} = 1$). One backward pass gives $\bar{\mathbf{f}}^\top J$, i.e. **one row of the Jacobian**: the gradient of one output w.r.t. *all* inputs (and all intermediate nodes).

$$\bar{v}_i = \sum_{j \,\in\, \text{children}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i} \qquad \text{(adjoints \textbf{add up} when a node feeds several consumers).}$$

Cost rule for $f : \mathbb{R}^n \to \mathbb{R}^m$ (Jacobian is $m \times n$)

Forward mode: n passes (one per input). Best when inputs are few and outputs many (2 design parameters → 200-sample pressure trace; initial condition → whole trajectory).
Reverse mode: m backward passes (one per output). Best when inputs are many and outputs few (NN with millions of weights → 1 loss; 200 blade coordinates → 1 efficiency). Costs memory (must store the forward values) and a backward pass is typically a small constant factor (about 1–4×) slower than the forward pass.
$n = m$: both need the same number of passes.

6.4 Rules for elementary operations

$v =$	forward tangent $\dot{v}$	reverse: contribution added to the inputs' adjoints
$a + b$	$\dot{a} + \dot{b}$	$\bar{a}$ += $\bar{v}$, $\bar{b}$ += $\bar{v}$
$a \cdot b$	$\dot{a}\,b + a\,\dot{b}$	$\bar{a}$ += $\bar{v}\,b$, $\bar{b}$ += $\bar{v}\,a$
a/b	$(\dot{a}b - a\dot{b})/b^2$	$\bar{a}$ += $\bar{v}/b$, $\bar{b}$ += $-\bar{v}\,a/b^2$
$\sin a$	$\cos a \cdot \dot{a}$	$\bar{a}$ += $\bar{v}\cos a$
$\cos a$	$-\sin a \cdot \dot{a}$	$\bar{a}$ += $-\bar{v}\sin a$
e^a	$e^a \dot{a}$	$\bar{a}$ += $\bar{v}\,e^a$
$\ln a$	$\dot{a}/a$	$\bar{a}$ += $\bar{v}/a$
a^n	$n a^{n-1}\dot{a}$	$\bar{a}$ += $\bar{v}\,n a^{n-1}$
$\sqrt{a}$	$\dot{a}/(2\sqrt{a})$	$\bar{a}$ += $\bar{v}/(2\sqrt{a})$
$\arcsin a$	$\dot{a}/\sqrt{1-a^2}$	$\bar{a}$ += $\bar{v}/\sqrt{1-a^2}$

Textbook example: $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \sin x_2$ **at** $(2, 5)$

Forward pass

v_0	x_1	2
v_1	x_2	5
v_2	$\ln v_0$	0.6931
v_3	$v_0 v_1$	10
v_4	$-\sin v_1$	0.9589
v_5	$v_2 + v_3$	10.6931
v_6	$v_5 + v_4$	$f = 11.6520$

Forward mode, seed $(\dot{x}_1, \dot{x}_2) = (1, 0)$

$\dot{v}_2$	$\dot{v}_0/v_0$	0.5
$\dot{v}_3$	$\dot{v}_0 v_1 + v_0 \dot{v}_1$	5
$\dot{v}_4$	$-\cos(v_1)\dot{v}_1$	0
$\dot{v}_6$	$\dot{v}_2 + \dot{v}_3 + \dot{v}_4$	$\partial f/\partial x_1 = 5.5$

Reverse mode, seed $\bar{v}_6 = 1$

$\bar{v}_6$		1
$\bar{v}_5, \bar{v}_4$	$\bar{v}_6 \cdot 1$	1, 1
$\bar{v}_3, \bar{v}_2$	$\bar{v}_5 \cdot 1$	1, 1
$\bar{v}_1$	$\bar{v}_3 v_0 + \bar{v}_4(-\cos v_1)$	$2 - 0.2837 = 1.7163$
$\bar{v}_0$	$\bar{v}_2/v_0 + \bar{v}_3 v_1$	$0.5 + 5 = 5.5$

One backward pass gives the whole gradient $\nabla f(2, 5) = (5.5,\ 1.7163)$. Forward mode needed a second pass with seed $(0, 1)$ for $\partial f/\partial x_2$. Note how x_1 and x_2 each feed two nodes, so their adjoints are sums of two contributions.

PS2 problem 1: $f_1 = x_1^2 + \sin x_2,\ f_2 = x_1 x_2 + e^{x_1}$ **at** $(2, \pi)$

Values: $f_1 = 4 + \sin\pi = 4$, $f_2 = 2\pi + e^2 \approx 13.672$.

$$J = \begin{bmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 & \cos x_2 \\ x_2 + e^{x_1} & x_1 \end{bmatrix} = \begin{bmatrix} 4 & -1 \\ \pi + e^2 \approx 10.531 & 2 \end{bmatrix}.$$

Forward mode: seed $\dot{\mathbf{x}} = [1, 0]^\top$ gives column 1 $= (4,\ 10.531)$; seed $[0, 1]^\top$ gives column 2 $= (-1,\ 2)$. **2 forward passes.** Reverse mode: seed $\bar{\mathbf{f}} = [1, 0]$ gives row 1 $= (4, -1)$; seed $[0, 1]$ gives row 2 $= (10.531, 2)$. **2 backward passes.** With 2 inputs and 2 outputs neither mode is cheaper in pass count; forward is slightly cheaper per pass (no stored graph).

Exercise: pump model, adjoints that cancel

At $(Q, n, \rho) = (4, 4, 3)$ with head $H = n^2/Q$ and power $P_h = \rho Q H$: in the P_h backward pass Q receives $+12$ via ρQ and -12 via n^2/Q, which cancel. Indeed $P_h = \rho Q \cdot n^2/Q = \rho n^2$ does not depend on Q. The full Jacobian of (P_h, η) w.r.t. (Q, n, ρ) is $\left(\begin{smallmatrix} 0 & 24 & 16 \\ 3/4 & -1/2 & 0 \end{smallmatrix}\right)$: 2 reverse passes (outputs) vs. 3 forward passes (inputs).

6.5 Finite differences

Key formulas

$$\text{forward:}\quad f'(x) \approx \frac{f(x+h)-f(x)}{h} \qquad \text{error } O(h)$$

$$\text{central:}\quad f'(x) \approx \frac{f(x+h)-f(x-h)}{2h} \qquad \text{error } O(h^2)$$

$$\text{complex step:}\quad f'(x) \approx \frac{\operatorname{Im} f(x+ih)}{h} \qquad \text{error } O(h^2),\ \text{no subtraction}$$

Too large h: truncation error. Too small h: round-off from subtracting two nearly equal numbers (subtractive cancellation). For forward differences in float64 the best $h \approx \sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$. The complex step has no subtraction, so $h = 10^{-20}$ works and the result is exact to machine precision (but the code must accept complex numbers). SciPy's SLSQP defaults to 2-point finite differences when no `jac` is given.

6.6 Pitfalls (favourite conceptual questions)

- **Discontinuities and branches.** AD differentiates only the branch that was executed and ignores how a parameter shifts *when* an event (a bounce, a contact, an `if`) happens. PS2 bouncing ball: the final velocity depends on the initial height h_0 only through the bounce timing, so AD reports $\partial v_{\text{final}}/\partial h_0 \approx 0$, while a finite-difference perturbation shows a real (jumpy) sensitivity. Always cross-check AD against finite differences around discontinuities. The sensitivities to g, e and v_0 change at each bounce (the restitution e multiplies the velocity at every impact).
- **`torch.where` with a bad branch.** Both branches are evaluated and differentiated; if the unused branch produces NaN/inf, the gradient is still NaN ($0 \cdot \text{NaN} = \text{NaN}$). Fix: sanitize the input of the bad branch.
- **Catastrophic cancellation.** Log-mean temperature difference $\Delta T_{lm} = (\Delta T_1 - \Delta T_2)/\ln(\Delta T_1/\Delta T_2)$ near $\Delta T_1 = \Delta T_2$: the value is accurate, but its derivative is two terms of size $\approx 50/\delta$ that cancel down to 0.5, so float32 gives wrong signs and magnitudes, and NaN at equality. Fix: a series expansion near the singular point ($\Delta T_{lm} \approx m(1 - u^2/3 - \dots)$). Switching to float64 only moves the problem closer to $\delta = 0$.
- **Vanishing / exploding gradients through long chains** (unrolled solvers, RNNs, logistic map $x_{j+1} = rx_j(1-x_j)$): the adjoint is a product of local derivatives, $\partial x_{40}/\partial x_k = \prod_{j=k}^{39} r(1-2x_j)$. Factors < 1 in size make it vanish exponentially (for $r = 2.5$ the factor at the fixed point $x^* = 0.6$ is -0.5); factors > 1 make it explode ($r = 3.9$: $\sim 6 \times 10^9$). Fixes: gradient clipping, LSTM-type units, shorter horizons.
- **Unrolled iterative solvers converge to the implicit derivative.** Kepler $E = M + e\sin E$ unrolled: the tangent $\dot{E}_k$ goes $1, 1.5, 1.75, \dots \to 1/(1 - e\cos E) = 2$ for $e = \frac{1}{2}$.
- **PyTorch mechanics.** Two `backward()` calls on one graph need `retain_graph=True` on the first; zero `.grad` in between or the rows add up.

PS2 problem 2: external fuel tank (ellipsoid)

Minimize drag $D = \frac{1}{2}\rho U^2 C_D S$ with $C_D = C_f\left[1 + 1.5(d/l)^{3/2} + 7(d/l)^3\right]$ and surface $S(l,d)$, subject to $V = \frac{\pi}{6}d^2 l \geq 0.5\,\text{m}^3$. Build the graph $v_1 = d/l,\ v_2 = v_1^2,\ v_3 = 1 - v_2,\ v_4 = \sqrt{v_3},\ v_5 = \arcsin v_4, \dots$ up to D, run the forward pass at $(l,d) = (2.5, 0.8)$, then the reverse pass from $\bar{D} = 1$ to get $\partial D/\partial l$ and $\partial D/\partial d$. Interpretation: increasing d grows both C_D (bluffer) and S, so drag is very sensitive to d; increasing l lowers d/l (less form drag) but adds wetted area. At the optimum the volume constraint is active, so $l = 3V_{\text{req}}/(\pi d^2)$ reduces the problem to one variable (a check for the SLSQP result). SciPy's `'ineq'` constraints mean $g(\mathbf{x}) \geq 0$, so pass $V(\mathbf{x}) - 0.5$.

7 Neural networks, autoencoders and least volume

7.1 Feed-forward networks (MLPs)

An L-layer MLP alternates affine maps and pointwise nonlinearities:

$$f_{\text{MLP}}(\mathbf{x}) = W_L\,\sigma\big(W_{L-1}\,\sigma(\cdots\sigma(W_1\mathbf{x} + \mathbf{b}_1)\cdots) + \mathbf{b}_{L-1}\big) + \mathbf{b}_L.$$

Without the nonlinearity σ the whole network collapses to one linear map. It is trained by minimizing a loss with (mini-batch) SGD or Adam, with the gradient computed by reverse-mode AD (backpropagation, §6).

Choice	What to know
ReLU $\max(0, x)$	Default. The network output is piecewise linear, visible as kinks or straight segments when only a few hidden units are used. Units can "die" (always 0).
LeakyReLU $\max(0.2x, x)$	Avoids dead units. Common in GAN discriminators.
tanh, sigmoid	Saturate for large $\lvert x\rvert$, so gradients vanish in deep stacks.
Output layer	Regression: no activation. Binary classification: raw logits with `BCEWithLogitsLoss` (numerically stable), or sigmoid with `BCELoss`. Never sigmoid followed by `BCEWithLogitsLoss`.
Regularization	L_2 weight decay, L_1, dropout, early stopping, batch norm, **spectral normalization** (divide each W by its largest singular value, so each layer is 1-Lipschitz: $\lVert f(\mathbf{x}) - f(\mathbf{x}')\rVert \le \lVert \mathbf{x} - \mathbf{x}'\rVert$).

Training loop: `opt.zero_grad()` → forward → `loss.backward()` → `opt.step()`. Forgetting `zero_grad` accumulates gradients across steps.

7.2 Autoencoders

An encoder E maps $\mathbf{x} \in \mathbb{R}^d$ to a low-dimensional code $\mathbf{z} = E(\mathbf{x}) \in \mathbb{R}^k$ (the bottleneck) and a decoder maps it back, $\hat{\mathbf{x}} = D(\mathbf{z})$. Train with the reconstruction loss $\frac{1}{N}\sum_i \lVert \mathbf{x}_i - D(E(\mathbf{x}_i))\rVert^2$.

PCA vs. autoencoder

A *linear* autoencoder (no activations) learns the same subspace as PCA. A nonlinear one can unroll curved manifolds: a 1-D curve embedded nonlinearly in 3-D needs all 3 PCA components (the explained variance is spread over all three), while an AE with a 1-D or 2-D latent can reconstruct it. The price: non-convex training.

Observation in the AE experiments	Explanation
Latent dim = true dim (1) often reconstructs badly	Non-convex training gets stuck in poor local minima; a few extra latent dimensions make optimization easier
Latent dim $\ge$ data dim reconstructs perfectly	The AE can learn a near-identity map; the bottleneck is gone, so nothing is learned about intrinsic dimension
Different result every run	Random initialization plus non-convex loss (PCA is deterministic up to sign)
Latent axes are not ordered or orthogonal	Nothing in the loss asks for it (PCA's eigenvalue ordering has no AE analogue)
1–2 hidden units: poor reconstruction with visible straight segments	Too little capacity; each ReLU unit adds one kink
L_1 penalty on $\mathbf{z}$ does not reveal the dimension	The encoder shrinks $\mathbf{z}$ while the decoder scales its weights up to compensate

7.3 Least volume (LV) regularization

Goal: an over-parameterized AE that automatically switches off unneeded latent dimensions, giving an ordered, PCA-like latent space.

Key formulas

$$\mathcal{L} = \underbrace{\frac{1}{N}\sum_i \lVert \mathbf{x}_i - D(E(\mathbf{x}_i))\rVert^2}_{\text{reconstruction}} + \lambda \underbrace{\Big(\prod_{j=1}^{k}(\sigma_j + \eta)\Big)^{1/k}}_{\text{volume}=\exp\left(\frac{1}{k}\sum_j \log(\sigma_j+\eta)\right)}, \qquad \sigma_j = \operatorname{std}(z_j) \text{ over the batch.}$$

- The geometric mean of the per-dimension standard deviations measures the volume the latent codes occupy. η avoids $\log 0$ once a dimension collapses.
- **Catch:** the encoder could shrink all of $\mathbf{z}$ uniformly while the decoder scales up its weights. **Fix: spectral normalization on the decoder** (Lipschitz ≤ 1), so the only way to reduce volume is to genuinely collapse dimensions.
- After training, sort latent dimensions by σ_j: the ones that did not collapse give the intrinsic dimension (automatic order selection). A linear LV autoencoder is equivalent to PCA.
- Diagnostics: bar plot of sorted latent stds, latent covariance matrix, parity plot (true vs. reconstructed values), pair plot of latent codes.

8 Generative models

A generative model learns the data distribution $p_{\text{data}}(\mathbf{x})$ well enough to *sample* new designs from it. Most models here are **push-forward** models: draw simple noise $\mathbf{z} \sim p(\mathbf{z})$ (usually $\mathcal{N}(0, I)$) and map it through a learned function G_θ to $\mathbf{x} = G_\theta(\mathbf{z})$. They differ in how G_θ is trained and whether they also give an inverse map $\mathbf{x} \to \mathbf{z}$ or a likelihood $p(\mathbf{x})$. The running toy example is the 2-D *ring of Gaussians* (several modes on a circle), later airfoils (192 points) and the Swiss roll.

8.1 Generative adversarial networks (GANs)

A generator G maps noise to samples; a discriminator $D(\mathbf{x}) \in (0, 1)$ estimates the probability that $\mathbf{x}$ is real. They play a minimax game:

Key formulas

$$\min_G \max_D \; V(D, G) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\big[\log D(\mathbf{x})\big] + \mathbb{E}_{\mathbf{z}\sim p_z}\big[\log\big(1 - D(G(\mathbf{z}))\big)\big].$$

Optimal discriminator and what the generator minimizes

For fixed G with sample density p_g, $V = \int p_{\text{data}} \log D + p_g \log(1 - D)\, d\mathbf{x}$. Maximizing $a \log y + b \log(1 - y)$ over y gives $y = a/(a + b)$, so

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}, \qquad V(D^*, G) = 2\,\text{JS}(p_{\text{data}} \,\|\, p_g) - \log 4.$$

So with a perfect discriminator the generator minimizes the Jensen–Shannon divergence. The global optimum is $p_g = p_{\text{data}}$, where $D^* \equiv \frac{1}{2}$. When the supports do not overlap, JS $= \log 2$ is constant and the generator's gradient vanishes (§10).

In practice each iteration takes one discriminator step and one generator step:

- **D step:** binary cross-entropy (BCE) with real samples labelled 1 and fakes labelled 0. Use `fake.detach()` so no gradient flows into G.
- **G step (non-saturating loss):** instead of minimizing $\log(1 - D(G(\mathbf{z})))$, which has a vanishing gradient when D confidently rejects fakes, maximize $\log D(G(\mathbf{z}))$, i.e. BCE of $D(G(\mathbf{z}))$ against the *real* label 1.
- **Equilibrium signature:** $D(\text{real}) \approx D(\text{fake}) \approx 0.5$, D loss $\approx 2 \log 2 \approx 1.386$ (sum of two BCE terms), G loss $\approx \log 2 \approx 0.693$. This is healthy only if the samples also look right and are diverse.
- **Conditional GAN:** concatenate the condition $\mathbf{c}$ (one-hot mode index, or a continuous radius) to the input of both G and D.

8.2 GAN training pitfalls and diagnostics

The standard diagnostic panel: training losses of D and G, mean $D(\text{real})$ and $D(\text{fake})$ over time, a sample-diversity metric over time, final losses, a scatter/pair plot of generated vs. real samples, and latent interpolations.

What you see	Diagnosis	Causes and remedies
D loss $\to 0$, $D(\text{real}) \to 1$, $D(\text{fake}) \to 0$; G loss keeps rising; samples stuck in a poor region	**Discriminator overpowers generator.** D separates perfectly, so $\log(1 - D(G))$ saturates and G gets no useful gradient	$\text{lr}_D \gg \text{lr}_G$, D much larger than G, lucky D initialization. Lower lr_D, spectral normalization on D, WGAN-GP, label smoothing, noise on D inputs
$D(\text{real}) \approx D(\text{fake}) \approx 0.5$ flat from early on; D loss constant ≈ 1.39; diversity near zero or spiky; samples collapsed onto one or a few points	**Generator overpowers discriminator**, which leads to **mode collapse.** D gives no informative signal, so G jumps to whatever point currently fools it	$\text{lr}_G \gg \text{lr}_D$, D too weak. Balance learning rates (TTUR with $\text{lr}_D \geq \text{lr}_G$), give D capacity, minibatch discrimination or diversity penalty
Losses oscillate; samples hop from mode to mode	Minimax cycling (mode hopping)	Lower learning rates, Adam with $\beta_1 = 0.5$, OT or Wasserstein loss
Latent interpolation: abrupt jumps, or every $\mathbf{z}$ gives the same $\mathbf{x}$	Degenerate or collapsed map	As above
Spectral norm on D: smoother D loss, slower and steadier training	D is Lipschitz-limited, cannot move its boundary suddenly	Too strong a constraint can make D too weak (blurrier coverage)

Why can't the generator recover when D overpowers it?

The game is solved by alternating local gradient steps. If D moves its decision boundary faster than G can follow, the fakes sit where $D \approx 0$ and $\log(1-D)$ is flat, so G's gradient is tiny. G has the capacity to represent the data but no signal pointing towards it.

PS2 "word bank" of GAN improvements

- **WGAN-GP:** replace BCE by the Wasserstein-1 distance estimated by a *critic* (no sigmoid): $\min_G \max_D \mathbb{E}[D(\mathbf{x})] - \mathbb{E}[D(G(\mathbf{z}))]$, plus a gradient penalty $\lambda \mathbb{E}\big[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2\big]$ on interpolates $\hat{\mathbf{x}}$ to keep D 1-Lipschitz. Stable, non-vanishing gradients.
- **Spectral normalization:** each weight matrix divided by its largest singular value, so D is 1-Lipschitz.
- **Minibatch discrimination / diversity penalty:** D sees batch statistics, or G is penalized for similar samples; fights mode collapse.
- **TTUR:** two time-scale update rule, separate learning rates (usually $\mathrm{lr}_D > \mathrm{lr}_G$).
- **Label smoothing:** real labels 0.9 instead of 1; prevents an overconfident D.
- **Feature matching:** G matches the mean of D's intermediate features on real vs. fake data, not just D's output.
- **Progressive growing / curriculum** (start simple, increase resolution) and **self-attention** (long-range structure).

8.3 Optimal transport generators

Think of two piles of sand: optimal transport (OT) finds the cheapest way to move one into the other.

$$W_p(\mu,\nu) = \Big(\inf_{\pi \in \Pi(\mu,\nu)} \int \|\mathbf{x}-\mathbf{y}\|^p \, d\pi(\mathbf{x},\mathbf{y})\Big)^{1/p},$$

where Π is the set of couplings (joint distributions) with the two marginals. Exact OT is expensive; adding an entropy term gives the **Sinkhorn divergence** (parameter `blur`: larger is smoother and cheaper, smaller is sharper). An "entropic OT GAN" trains the generator by directly minimizing the Sinkhorn divergence between a generated batch and a real batch, *with no discriminator*.

Advantages: no minimax game (stable, one learning rate), good mode coverage (every real point must receive mass), meaningful gradients even when the distributions do not overlap, the loss value is interpretable. Drawbacks: `blur` tuning, $O(n^2)$ pairwise cost, less discriminative when the distributions start far apart (often used as a fine-tuning step after alignment). Like a GAN, it gives no inverse map $\mathbf{x} \to \mathbf{z}$.

8.4 Variational autoencoders (VAEs)

The encoder outputs a *distribution* over codes, $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \operatorname{diag}\boldsymbol{\sigma}^2_\phi(\mathbf{x}))$; the decoder defines $p_\theta(\mathbf{x}|\mathbf{z})$; the prior is $p(\mathbf{z}) = \mathcal{N}(0, I)$. Training maximizes the evidence lower bound (derivation in §11):

Key formulas

$$\log p_\theta(\mathbf{x}) \;\geq\; \mathrm{ELBO} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x}|\mathbf{z})\big]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{regularization}},$$

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag}\boldsymbol{\sigma}^2) \,\|\, \mathcal{N}(0,I)\big) = \tfrac{1}{2}\sum_{i=1}^{d}\big(\mu_i^2 + \sigma_i^2 - \log\sigma_i^2 - 1\big), \qquad \mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon},\ \boldsymbol{\epsilon} \sim \mathcal{N}(0,I).$$

With a Gaussian decoder of fixed variance, the reconstruction term is $\frac{1}{2\sigma_\theta^2}\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ up to a constant, so the loss is MSE + (weighted) KL. The **reparameterization trick** moves the randomness into $\boldsymbol{\epsilon}$, so gradients flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

- The KL term pulls every $q_\phi(\mathbf{z}|\mathbf{x})$ towards $\mathcal{N}(0,I)$, so the latent space has no holes: interpolation and sampling from the prior are meaningful.
- Strengths: an encoder (inverse map), stable training, interpretable smooth latent space, good mode coverage.
- Weaknesses: blurry samples (MSE averages), **posterior collapse** (KL dominates, the decoder ignores $\mathbf{z}$), the Gaussian assumption is restrictive.
- β-VAE: weight β on KL. Larger β gives a smoother, more disentangled latent space but worse reconstructions.

8.5 Normalizing flows

An invertible, differentiable f maps $\mathbf{z} \sim p_Z$ to $\mathbf{x} = f(\mathbf{z})$. Probability mass is conserved, so densities change by the volume change (Jacobian determinant):

Change of variables

$$p_X(\mathbf{x}) = p_Z\big(f^{-1}(\mathbf{x})\big)\left|\det \frac{\partial f^{-1}(\mathbf{x})}{\partial \mathbf{x}}\right| = p_Z(\mathbf{z})\left|\det \frac{\partial f(\mathbf{z})}{\partial \mathbf{z}}\right|^{-1}, \qquad \log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) - \sum_{\ell=1}^{K} \log\left|\det J_{f_\ell}(\mathbf{z}_{\ell-1})\right|$$

for a stack of K layers. Train by maximizing the exact log-likelihood of the data.

1-D intuition

$x = f(z) = \tanh(z)$ with $z \sim \mathcal{N}(0,1)$: $p_X(x) = p_Z(\operatorname{artanh} x)\,\frac{1}{1-x^2}$. Where f compresses space ($|f'|$ small, near $x = \pm 1$) the density piles up; where it stretches, the density thins out.

Layer	Map	$\log\lvert\det J\rvert$
Affine	$f(\mathbf{z}) = A\mathbf{z} + \mathbf{b}$	$\log\lvert\det A\rvert$ (rotation: 0, volume preserving)
Planar	$f(\mathbf{z}) = \mathbf{z} + \mathbf{u}\,h(\mathbf{w}^\top\mathbf{z} + b)$	$\log\lvert 1 + \mathbf{u}^\top\boldsymbol{\psi}(\mathbf{z})\rvert$, $\boldsymbol{\psi} = h'(\mathbf{w}^\top\mathbf{z} + b)\,\mathbf{w}$ (matrix determinant lemma)
Radial	$f(\mathbf{z}) = \mathbf{z} + \beta\frac{\mathbf{z}-\mathbf{z}_0}{\alpha+r}$, $r = \lVert\mathbf{z} - \mathbf{z}_0\rVert$	$(D-1)\log(1+h) + \log(1+h+h'r)$, $h = \frac{\beta}{\alpha+r}$; creates or fills holes around $\mathbf{z}_0$
Affine coupling (RealNVP)	$\mathbf{y}_1 = \mathbf{z}_1,\ \ \mathbf{y}_2 = \mathbf{z}_2 \odot e^{s(\mathbf{z}_1)} + t(\mathbf{z}_1)$	$\sum_j s_j(\mathbf{z}_1)$ (triangular Jacobian)

Coupling layers are the workhorse: s and t can be arbitrary networks, yet the inverse is trivial, $\mathbf{z}_2 = (\mathbf{y}_2 - t(\mathbf{y}_1)) \odot e^{-s(\mathbf{y}_1)}$, and the Jacobian $\begin{pmatrix} I & 0 \\ \partial\mathbf{y}_2/\partial\mathbf{z}_1 & \operatorname{diag}(e^{s(\mathbf{z}_1)}) \end{pmatrix}$ is triangular, so its determinant is the product of the diagonal. Alternate which half is transformed.

Strengths: exact likelihood, exact inverse (no separate encoder), sharp samples. Limitations: $\dim\mathbf{z} = \dim\mathbf{x}$ is forced, architecture must stay invertible, Jacobians can be expensive in high dimensions, memory grows with depth.

8.6 Continuous normalizing flows (neural ODEs)

Replace a stack of discrete layers by an ODE $\frac{d\mathbf{z}}{dt} = \mathbf{v}_\theta(\mathbf{z}, t)$. In the limit of infinitely many small layers the log-determinant becomes a trace (*instantaneous change of variables*):

Key formulas

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}_\theta}{\partial \mathbf{z}}\right) = -\nabla\cdot\mathbf{v}_\theta, \qquad \log p(\mathbf{x}) = \log p(\mathbf{z}_0) - \int_0^T \operatorname{tr}\left(\nabla_{\mathbf{z}}\mathbf{v}_\theta(\mathbf{z}(t), t)\right) dt.$$

A divergence-free field (pure rotation) keeps the density constant; a contracting field (negative divergence) increases it. Training needs an ODE solver plus backpropagation through it and the trace, which is expensive and hard in high dimensions. $\mathbf{v}_\theta$ should be Lipschitz (e.g. spectral normalization) so ODE solutions are unique.

8.7 Score matching and Langevin sampling

The **score** is the gradient of the log-density, a vector field pointing uphill towards high density:

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) \qquad \text{(independent of the normalizing constant, since } \nabla\log(\tilde{p}/Z) = \nabla\log\tilde{p}\text{)}.$$

Learn $\mathbf{s}_\theta(\mathbf{x}) \approx \nabla \log p_{\text{data}}(\mathbf{x})$:

Key formulas

Fisher divergence (intractable): $J(\theta) = \mathbb{E}_{p_{\text{data}}}\lVert\nabla\log p_{\text{data}}(\mathbf{x}) - \mathbf{s}_\theta(\mathbf{x})\rVert^2$

Hyvärinen score matching: $J(\theta) \propto \mathbb{E}_{p_{\text{data}}}\left[\operatorname{tr}(\nabla_{\mathbf{x}}\mathbf{s}_\theta(\mathbf{x})) + \frac{1}{2}\lVert\mathbf{s}_\theta(\mathbf{x})\rVert^2\right]$ (needs a trace: expensive)

Denoising score matching (DSM): $J_\sigma(\theta) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}},\ \boldsymbol{\epsilon}\sim\mathcal{N}(0,\sigma^2 I)}\left\lVert\mathbf{s}_\theta(\mathbf{x}+\boldsymbol{\epsilon}) + \frac{\boldsymbol{\epsilon}}{\sigma^2}\right\rVert^2$

Where the DSM target comes from

Perturb the data with Gaussian noise, $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$. Matching the score of the noisy distribution $p_\sigma(\tilde{\mathbf{x}})$ is equivalent (up to a constant) to matching the score of the known noise kernel: $\log p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = C - \frac{1}{2\sigma^2}\|\tilde{\mathbf{x}} - \mathbf{x}\|^2 \Rightarrow \nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = -\frac{\tilde{\mathbf{x}}-\mathbf{x}}{\sigma^2} = -\frac{\boldsymbol{\epsilon}}{\sigma^2}$. So the network learns to point from the noisy sample back to the clean one. No trace, cheap to train. It learns the score of the *blurred* density $p_\sigma = p_{\text{data}} * \mathcal{N}(0, \sigma^2 I)$, not of p_{data} itself.

Effect of the noise level σ (HS2025 review problem 2)

small σ (0.1)	Accurate, sharp score only very close to the data (arrows snap onto each ring). Far from the data there were no training samples, so the field is unlearned: tiny, random or chaotic arrows, messy streamlines that end on many separate points.
medium σ (0.5)	Points towards the rings from further away while still separating inner and outer ring. Best compromise.
large σ (1.0)	Smooth, global, nearly radial field pointing to the overall centre of mass. The rings are blurred together and fine structure is lost; streamlines flow cleanly inward to one central region.

Rule: noise of scale σ decides *where* training samples exist (a band of width $\approx \sigma$ around the data) and *how much* the learned density is blurred. Small σ: accurate but blind far away. Large σ: global guidance but biased. Diffusion models fix this by using many noise levels.

Langevin dynamics turns a score into samples:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha\, \mathbf{s}_\theta(\mathbf{x}_k) + \sqrt{2\alpha}\, \mathbf{z}_k, \qquad \mathbf{z}_k \sim \mathcal{N}(0, I) \qquad \big(\text{discretizes } d\mathbf{x}_t = \mathbf{s}(\mathbf{x}_t)\, dt + \sqrt{2}\, d\mathbf{W}_t\big).$$

Without the noise term this is gradient ascent on $\log p$ and every sample collapses onto the nearest mode. With the noise, as $\alpha \to 0$ and $k \to \infty$ the samples follow p exactly.

8.8 Diffusion models

Forward (noising) process, with a schedule β_t, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$ (so $\bar{\alpha}_t \to 0$ for large t):

Key formulas

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I); \qquad \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\boldsymbol{\epsilon}}{\sqrt{1 - \bar{\alpha}_t}}, \qquad \mathbf{s}_\theta = -\frac{\boldsymbol{\epsilon}_\theta}{\sqrt{1 - \bar{\alpha}_t}}.$$

$$\text{Training loss: } L(\theta) = \mathbb{E}_{\mathbf{x}_0,\, t \sim U\{1..T\},\, \boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon},\ t\big) - \boldsymbol{\epsilon} \big\|^2$$

$$\text{Reverse step: } \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\Big) + \sigma_t \mathbf{z}.$$

- It is DSM at many noise levels at once, with time t as an extra network input. Large t (heavy noise) teaches the global layout, small t the fine detail. This fixes the single-σ dilemma above.
- Predicting the noise $\boldsymbol{\epsilon}$ instead of the score carries the same information but gives better-scaled, more stable targets.
- The continuous view is a reverse-time SDE whose drift contains the score: $d\mathbf{x} = [f(\mathbf{x}, t) - g(t)^2 \nabla \log p_t(\mathbf{x})]\, dt + g(t)\, d\bar{\mathbf{W}}$ (sign conventions vary). Sampling is annealed Langevin dynamics.
- Strengths: high quality, stable training (plain regression), excellent mode coverage. Weakness: slow sampling (many network evaluations).

8.9 Flow matching

The deterministic, noise-free relative: learn a velocity field and integrate an ODE.

Key formulas

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{x}_0 \sim \mathcal{N}(0, I),\ \mathbf{x}_1 \sim p_{\text{data}}, \qquad \frac{d\mathbf{x}_t}{dt} = \mathbf{x}_1 - \mathbf{x}_0,$$

$$L(\theta) = \mathbb{E}_{t \sim U[0,1],\, \mathbf{x}_0, \mathbf{x}_1} \big\| \mathbf{v}_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0) \big\|^2, \qquad \text{sample: } \mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mathbf{v}_\theta(\mathbf{x}_t, t)\, \Delta t \ \text{ from } t = 0 \text{ to } 1.$$

- Training is *simulation-free* regression: no ODE solve, no trace, no backprop through a solver (unlike CNFs).
- The learned field is the *average* velocity over all pairs passing through a point, so sampling paths are curved even though each training path is straight (random pairs cross). Fewer Euler steps give coarser samples.

- Conditioning: feed $\mathbf{c}$ to $\mathbf{v}_\theta$.

8.10 Latent generative models

If the data lies on a low-dimensional manifold in a high-dimensional space (Swiss roll: 2-D sheet in 3-D; airfoils), modelling the raw space is hard. Instead: (1) train an autoencoder (e.g. least-volume AE, §7) to get a compact latent space; (2) train a GAN or flow-matching model on the latent codes; (3) sample latents and decode. The latent distribution is simpler, so training is faster and samples are better. Quality is limited by the autoencoder.

8.11 Which model when

Model	Training	Inverse $\mathbf{x} \to \mathbf{z}$	Likelihood	Coverage	Sampling	Main weakness
GAN	minimax	no	no	mode collapse risk	1 pass	unstable training
OT generator	Sinkhorn loss	no	no	good	1 pass	cost, `blur` tuning
VAE	ELBO	yes (approx.)	lower bound	good	1 pass	blurry, posterior collapse
Normalizing flow	max. likelihood	yes (exact)	exact	good	1 pass	architecture constraints
CNF	likelihood via ODE	yes	exact	good	ODE solve	expensive trace and backprop
Diffusion / score	DSM regression	via ODE	approx.	very good	many steps	slow sampling
Flow matching	velocity regression	via ODE	via ODE	very good	ODE steps	needs several steps
Latent model	AE + generator	yes	depends	good	fast	limited by AE

Need an encoder: VAE or flow. Need exact densities (anomaly detection, uncertainty): flow. Need fast sharp samples: GAN or latent model. Need diversity and quality and can afford time: diffusion or flow matching.

9 Transformers and models for sets

9.1 Self-attention

Input: a set or sequence of n tokens $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]$, $\mathbf{x}_i \in \mathbb{R}^d$ (e.g. points of a 2-D shape or 3-D scan). Each token produces a query, a key and a value with learned matrices:

Key formulas

$$\mathbf{q}_i = W_Q \mathbf{x}_i, \quad \mathbf{k}_i = W_K \mathbf{x}_i, \quad \mathbf{v}_i = W_V \mathbf{x}_i; \qquad \mathbf{z}_i = \sum_{j=1}^{n} a_{ij}\, \mathbf{v}_j,$$

$$a_{ij} = \operatorname{softmax}_j\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}\right) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d_k})}{\sum_l \exp(\mathbf{q}_i^\top \mathbf{k}_l/\sqrt{d_k})}.$$

Matrix form: $\operatorname{Attention}(Q,K,V) = \operatorname{softmax}\left(QK^\top/\sqrt{d_k}\right) V$.

Intuition

Token i asks a question ($\mathbf{q}_i$); every token j advertises what it has ($\mathbf{k}_j$). Their similarity decides how much of j's content ($\mathbf{v}_j$) token i takes. Each row of the attention matrix $A = [a_{ij}]$ is a probability distribution (non-negative, sums to 1). Dividing by $\sqrt{d_k}$ keeps dot products from growing with dimension and saturating the softmax.

- **Permutation equivariance:** without positional encodings, shuffling the input tokens shuffles the outputs the same way. Adding a symmetric pooling (mean, max, or a learned CLS token that attends to all points) makes the final prediction **permutation invariant**, which is exactly what unordered point clouds need. Ordered sequences (text, time series) need positional encodings.

- **Cost:** the attention matrix is $n \times n$, so compute and memory are $O(n^2)$ in the number of tokens.
- **Padding masks:** to batch sets of different sizes, pad to $N_{\max}$ and set the logits of padded tokens to $-\infty$ so they get zero attention.
- **Multi-head attention:** several attention maps in parallel, each with its own W_Q, W_K, W_V, concatenated. A transformer block = attention + MLP, each with a residual connection and layer norm.
- **Reading attention maps:** a bright a_{ij} means token i draws heavily on token j. Attention concentrated on corners or edges shows which geometric cues the classifier uses.

9.2 MLP vs. PointNet vs. set transformer

Model	Strengths	Weaknesses
MLP on flattened points $[p_{1,x}, p_{1,y}, \dots]$	simple; fast and accurate when the number and order of points are fixed	not permutation invariant: breaks under reordering, variable length, padding; no notion of geometry
PointNet: $\mathbf{h}_i = g(\mathbf{p}_i)$ (shared MLP), $\mathbf{z} = \max_i \mathbf{h}_i$, $y = f(\mathbf{z})$	permutation invariant by construction; handles variable point counts; few parameters; good for coarse global shape classification	points never interact before pooling, so relations (distances between features, corners, joint configurations) need very wide layers; pooling discards spatial structure
Set transformer (self-attention on points)	every point attends to every other, so it captures edges, corners and long-range relations; robust to noise, order and set size	$O(n^2)$ cost; more hyperparameters (depth, width, heads)

10 Measuring distances between distributions

Generative models are trained by making a model distribution q close to the data p. *Which* notion of "close" you use decides the gradients you get and therefore what the model learns.

Key formulas

Forward KL: $\mathrm{KL}(p \,\|\, q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx = \mathbb{E}_p[\log p - \log q]$

Reverse KL: $\mathrm{KL}(q \,\|\, p) = \int q(x) \log \frac{q(x)}{p(x)}\, dx = -H(q) - \mathbb{E}_q[\log p]$

Jensen–Shannon: $\mathrm{JS}(p,q) = \frac{1}{2}\mathrm{KL}(p \,\|\, m) + \frac{1}{2}\mathrm{KL}(q \,\|\, m), \quad m = \frac{1}{2}(p+q), \quad 0 \le \mathrm{JS} \le \log 2$

MMD: $\mathrm{MMD}^2(p,q) = \mathbb{E}_{x,x' \sim p} k(x,x') - 2\,\mathbb{E}_{x \sim p,\, y \sim q} k(x,y) + \mathbb{E}_{y,y' \sim q} k(y,y'), \quad k(x,y) = e^{-\frac{(x-y)^2}{2\sigma_k^2}}$

Wasserstein-2: $W_2^2(p,q) = \inf_{\pi \in \Pi(p,q)} \int \|x - y\|^2 \, d\pi(x,y) \overset{\text{1-D}}{=} \int_0^1 \left|F_p^{-1}(u) - F_q^{-1}(u)\right|^2 du$

In 1-D the optimal transport plan simply sorts both sample sets and pairs quantiles.

10.1 Closed forms for Gaussians

$$\mathrm{KL}\big(\mathcal{N}(\mu_p, \sigma_p^2) \,\|\, \mathcal{N}(\mu, \sigma_q^2)\big) = \log \frac{\sigma_q}{\sigma_p} + \frac{\sigma_p^2 + (\mu_p - \mu)^2}{2\sigma_q^2} - \frac{1}{2}, \qquad \frac{\partial \mathrm{KL}}{\partial \mu} = \frac{\mu - \mu_p}{\sigma_q^2}.$$

Equal variances: $\mathrm{KL} = \frac{(\mu_p - \mu)^2}{2\sigma^2}$, unbounded as the means separate. $W_2^2 = (\mu_p - \mu)^2 + (\sigma_p - \sigma_q)^2$. Entropy $H(\mathcal{N}(\mu, \sigma^2)) = \frac{1}{2}\log(2\pi e \sigma^2)$. Against a Gaussian model q, the forward KL sees any target p only through its mean and variance: $\mathrm{KL}(p\|q) = -H(p) + \log(\sigma_q\sqrt{2\pi}) + \frac{\mathrm{Var}_p[x] + (\mathbb{E}_p[x] - \mu)^2}{2\sigma_q^2}$.

10.2 Properties

Distance	Symm.	Bounded	Finite, disjoint supports	Metric	Gradient behaviour
Forward KL$(p\|\|q)$	no	no	no $(p>0, q=0 \Rightarrow \infty)$	no	**mode covering**: q must put mass wherever p has mass $\Rightarrow$ wide q spanning all modes (and the gap between them)
Reverse KL$(q\|\|p)$	no	no	no $(q>0, p=0 \Rightarrow \infty)$	no	**mode seeking**: q is punished for mass where $p \approx 0$, not for missing modes $\Rightarrow$ locks onto one mode
JS	yes	yes, $\leq \log 2$	yes (stuck at $\log 2$)	$\sqrt{\text{JS}}$ is	bounded and soft, but **flat when far apart** (vanishing gradient: the original GAN problem)
MMD (RBF)	yes	yes	yes	yes	bandwidth σ_k: small = local detail, flat for distant distributions; large = coarse, blurs detail
W_2	yes	no	**yes**, grows with distance	yes	informative everywhere: gradient proportional to how far mass must move

Fitting one Gaussian to a bimodal target

Target $p = 0.35\,\mathcal{N}(-2.5, 0.6^2) + 0.65\,\mathcal{N}(2.5, 0.6^2)$, model $q = \mathcal{N}(\mu, \sigma^2)$.

- **Minimizing forward KL** gives moment matching: $\mu = \mathbb{E}_p[x] = 0.75$, $\sigma^2 = \text{Var}_p[x]$ (large). A wide Gaussian covering both modes, with most mass in the empty gap.
- **Minimizing reverse KL** $= -H(q) - \mathbb{E}_q[\log p]$ gives a narrow Gaussian on *one* mode, $\mu \approx 2.5$, $\sigma \approx 0.6$. The heavier mode (65 %) wins because $\log p$ is larger there. A q that straddles both modes pays heavily for the mass it puts in the gap.

Non-overlapping supports

Model shifted completely off the target: KL is infinite (or a huge constant set by the numerical EPS, with near-zero gradient), JS sits at $\log 2$ with zero gradient, MMD with a small bandwidth is nearly constant. Only Wasserstein still says "move this way, and here is how far". This is why OT generators and WGANs fix vanishing GAN gradients, and why variational inference (reverse KL) underestimates spread.

11 Inference: Bayesian regression, MCMC and variational inference

Instead of one best weight vector, Bayesian inference keeps a distribution over weights, which gives predictions with uncertainty:

Key formulas

$$\underbrace{p(\mathbf{w} \mid D)}_{\text{posterior}} = \frac{\overbrace{p(D \mid \mathbf{w})}^{\text{likelihood}}\ \overbrace{p(\mathbf{w})}^{\text{prior}}}{\underbrace{p(D)}_{\text{evidence}}}, \qquad p(D) = \int p(D \mid \mathbf{w})\, p(\mathbf{w})\, d\mathbf{w}, \qquad p(y_* \mid x_*, D) = \int p(y_* \mid x_*, \mathbf{w})\, p(\mathbf{w} \mid D)\, d\mathbf{w}.$$

The evidence integral is the hard part: it is intractable for anything but simple models.

11.1 Bayesian linear regression (closed form)

Model $y = \mathbf{w}^\top \boldsymbol{\phi}(x) + \epsilon$, noise $\epsilon \sim \mathcal{N}(0, \beta^{-1})$ (precision $\beta = 1/\sigma^2$), prior $p(\mathbf{w}) = \mathcal{N}(0, \alpha^{-1} I)$, design matrix Φ. Gaussian prior times Gaussian likelihood gives a Gaussian posterior:

Key formulas

$$p(\mathbf{w} \mid D) = \mathcal{N}(\mathbf{m}_N, S_N), \qquad S_N = (\alpha I + \beta \Phi^\top \Phi)^{-1}, \qquad \mathbf{m}_N = \beta S_N \Phi^\top \mathbf{y},$$

$$p(y_* \mid x_*, D) = \mathcal{N}\Big(\mathbf{m}_N^\top \boldsymbol{\phi}(x_*),\ \ \sigma_N^2(x_*) = \underbrace{\beta^{-1}}_{\text{noise}} + \underbrace{\boldsymbol{\phi}(x_*)^\top S_N \boldsymbol{\phi}(x_*)}_{\text{parameter uncertainty}}\Big).$$

- The posterior mean (MAP) equals ridge regression with $\lambda = \alpha/\beta$: a **Gaussian prior is an L_2 penalty** (a Laplace prior is an L_1 penalty), since $-\log p(\mathbf{w}|D) = \frac{\beta}{2}\|\mathbf{y} - \Phi\mathbf{w}\|^2 + \frac{\alpha}{2}\|\mathbf{w}\|^2 + \text{const}$.
- Predictive bands are narrow near the data and widen away from it (parameter uncertainty grows). More data shrinks S_N; the noise floor β^{-1} remains.
- A linear model on nonlinear data ($y = \sin x$) has bands that are confidently wrong: the uncertainty is only as good as the model.

11.2 Why we need approximations

For a neural network $f(x; \mathbf{w}) = \mathbf{w}_2^\top \sigma(\mathbf{w}_1 x + \mathbf{b}_1) + b_2$ the posterior is non-Gaussian, multimodal (hidden units can be permuted) and high-dimensional; the evidence integral has no closed form. Plain Monte Carlo (sample from the prior, weight by the likelihood) or importance sampling with a poor proposal fails: almost no samples land in the narrow region of high posterior mass (think of a curved "banana"), so a few samples get all the weight.

11.3 MCMC: Metropolis–Hastings

Build a Markov chain whose stationary distribution is the posterior; you only need the posterior up to a constant.

Key formulas

1. Start at $\mathbf{w}^{(0)}$.
2. Propose $\mathbf{w}^* \sim q(\mathbf{w}^* \mid \mathbf{w}^{(t)})$, e.g. $\mathbf{w}^* = \mathbf{w}^{(t)} + s\,\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ (symmetric).
3. Accept with probability $\alpha = \min\Big(1,\ \dfrac{p(Y|X,\mathbf{w}^*)\,p(\mathbf{w}^*)}{p(Y|X,\mathbf{w}^{(t)})\,p(\mathbf{w}^{(t)})}\Big)$ (the evidence cancels). If rejected, repeat the current sample.
4. Repeat; discard the burn-in; use the remaining samples for predictions.

- **Step size s:** too small ⇒ acceptance near 100 % but tiny moves, slow random-walk mixing, strongly correlated samples. Too large ⇒ most proposals rejected, the chain sticks. Aim for moderate acceptance (roughly 20–50 %). Check trace plots.
- **Hamiltonian Monte Carlo (HMC)** uses $\nabla \log p$ (from autodiff) to simulate a particle with momentum, making long, informed moves with high acceptance. Tune the leapfrog step size ε (too large: energy errors, rejections) and the number of steps L. NUTS tunes L automatically.
- MCMC is asymptotically exact but slow; use it as the reference to validate faster approximations.

11.4 Variational inference (VI)

Choose a tractable family $q_\phi(\mathbf{w})$, e.g. **mean-field Gaussian** $q_\phi = \mathcal{N}(\boldsymbol{\mu}_\phi, \operatorname{diag}\boldsymbol{\sigma}_\phi^2)$, and optimize ϕ so that q_ϕ is close to the posterior.

The ELBO from Jensen's inequality

$$\log p(Y|X) = \log \int q_\phi(\mathbf{w})\,\frac{p(Y|X,\mathbf{w})\,p(\mathbf{w})}{q_\phi(\mathbf{w})}\,d\mathbf{w} \ \geq\ \mathbb{E}_{q_\phi}\Big[\log \frac{p(Y|X,\mathbf{w})\,p(\mathbf{w})}{q_\phi(\mathbf{w})}\Big]$$

because log is concave ($\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log \cdot]$). Rearranging:

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi}[\log p(Y|X,\mathbf{w})] - \mathrm{KL}\big(q_\phi(\mathbf{w}) \,\|\, p(\mathbf{w})\big), \qquad \log p(Y|X) - \mathcal{L}(\phi) = \mathrm{KL}\big(q_\phi(\mathbf{w}) \,\|\, p(\mathbf{w}|X,Y)\big) \geq 0.$$

So maximizing the ELBO is the same as minimizing the **reverse KL** from q to the true posterior.

- Consequence of reverse KL (§10): VI is mode seeking and tends to **underestimate the posterior variance**. Mean-field ignores correlations between weights, so it cannot fit a curved banana-shaped posterior.
- **Score-function (REINFORCE / log-derivative) estimator:** $\nabla_\phi \mathbb{E}_{q_\phi}[f(\mathbf{w})] = \mathbb{E}_{q_\phi}[f(\mathbf{w})\,\nabla_\phi \log q_\phi(\mathbf{w})]$, using $\nabla \log q = \nabla q / q$. Works for any q and any f (black-box VI), but has *high variance*.

- **Reparameterization trick:** write $\mathbf{w} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$. The expectation is now over $\boldsymbol{\epsilon}$, which does not depend on ϕ, so gradients flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ directly: *low variance*. This is what VAEs use.
- VI gives an approximate posterior in seconds instead of minutes. A VAE is *amortized* VI: one encoder network outputs $q(\mathbf{z}|\mathbf{x})$ for every $\mathbf{x}$.

Effect of experiment knobs

More hidden units: more flexible, wider uncertainty away from the data. More training data: narrower posterior and predictive bands. Stronger prior (larger α, smaller prior variance): weights shrink, functions become smoother, possibly underfit; a too-weak prior lets the model overfit with overconfident bands.

11.5 Probabilistic programming (Pyro)

Write the generative story as code: a `model` with priors (`pyro.sample("E", dist.Normal(...))`) and the likelihood of the observations (`pyro.sample("obs", ..., obs=y)`), plus a `guide` (the variational family q). Then run SVI (maximizing the ELBO) or MCMC (NUTS) without deriving anything by hand.

Material identification with the Ramberg–Osgood law

$\varepsilon = \dfrac{\sigma}{E} + \alpha\left(\dfrac{\sigma}{\sigma_0}\right)^n$ with unknowns E (Young's modulus), σ_0 (yield stress), α, n (hardening exponent), from noisy stress–strain data. Priors encode physical ranges (positive, typical orders of magnitude). The posterior gives credible intervals. Wide marginals or strong correlations between α and n (or σ_0) mean the data cannot separate them: they are poorly identifiable. Collect more data in the plastic region, or tighten the priors.

12 Reinforcement learning

12.1 Setting

An agent observes state s_t, takes action a_t from its policy $\pi(a|s)$, receives reward r_t and moves to s_{t+1}. It maximizes the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad 0 \le \gamma < 1.$$

γ close to 1: far-sighted but slower and noisier learning; close to 0: myopic. Value functions: $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ and $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$. Bellman optimality: $Q^*(s,a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^*(s',a')\big]$. Course example: pendulum swing-up (continuous angle and angular velocity; torque actions).

12.2 The four methods

Deep Q-Network (DQN): value-based

$$a_t = \arg\max_a Q_\theta(s_t, a) \;\; (\epsilon\text{-greedy}), \qquad y_t = r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a'), \qquad \mathcal{L}_{TD} = \big(Q_\theta(s_t, a_t) - y_t\big)^2.$$

A network predicts Q for each discrete action. The Bellman target acts as a supervised label (temporal-difference learning). With probability ϵ take a random action (exploration). Mini-batches are drawn from a **replay buffer** of past transitions (breaks correlations, reuses data); a slowly updated **target network** stabilizes the moving target. Needs discrete actions.

REINFORCE: policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(a|s)\, R(\tau)\big].$$

The same log-derivative trick as the score-function estimator in VI. Raise the log-probability of actions in episodes with high return. Handles continuous actions and stochastic policies naturally, but the Monte Carlo return makes the gradient **very high variance**, so learning is slow and noisy.

Actor–critic

$$y_t = r_t + \gamma V_\phi(s_{t+1}), \qquad A_t = y_t - V_\phi(s_t), \qquad \mathcal{L}_{\text{critic}} = (V_\phi(s_t) - y_t)^2, \qquad \text{actor step} \propto \nabla_\theta \log \pi_\theta(a_t|s_t)\, A_t.$$

The critic V_ϕ is a learned baseline. The **advantage** A_t asks "was this action better than expected in this state?": positive ⇒ make it more likely, negative ⇒ less likely. Subtracting the baseline lowers variance without biasing the gradient.

Proximal policy optimization (PPO)

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \qquad \mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,A_t\big)\Big].$$

Clipping removes the incentive to move the policy more than a factor $1 \pm \epsilon$ (typically $\epsilon = 0.2$) per update, so one bad batch cannot wreck the policy. Robust, the default in robotics and continuous control.

Method	Type	Actions	Pros	Cons
DQN	value-based, off-policy	discrete	sample-efficient (replay)	unstable, discrete actions only
REINFORCE	policy gradient, on-policy	discrete or continuous	simple, optimizes the policy directly	high-variance gradients, slow
Actor–critic	policy + learned value	both	lower variance via advantage	hyperparameter sensitive
PPO	clipped actor–critic	both	stable and robust	still needs tuning

Knobs and diagnostics

ϵ (exploration): too low gets stuck in a poor policy, too high behaves randomly. Learning rate: too high makes learning curves collapse after rising. γ: horizon. Entropy bonus: keeps the policy stochastic (more exploration). PPO clip ϵ: maximum policy change per update. Diagnostics: learning curve of episode return (noisy; look at the smoothed trend), heat maps of V or Q over (angle, velocity), policy maps (chosen torque per state).

13 Active learning and semi-supervised learning (small data)

In engineering, labels (experiments, high-fidelity simulations) are expensive while unlabelled inputs are cheap. Two complementary ideas: **active learning** chooses *which* point to label next; **semi-supervised learning** uses the unlabelled points too.

13.1 Active learning strategies

Strategy	Picks the point with	Pro	Con
Uncertainty sampling	highest predictive uncertainty: least confidence $(1 - \max_c p_c)$, smallest margin $(p_{(1)} - p_{(2)})$, highest entropy $(-\sum_c p_c \log p_c)$, largest GP variance	simple, focuses on the decision boundary	chases outliers and noise, ignores how representative a point is
Query-by-committee	most disagreement among an ensemble	works with any model type	must train many models
Expected model change	largest expected change in parameters	targets impactful points	expensive
Expected error reduction	largest expected drop in generalization error	the most "natural" objective	very expensive (retrain per candidate)
Variance reduction	largest expected drop in output variance	good for uncertainty quantification	expensive
Density-weighted	uncertain *and* representative of the data distribution	avoids outliers	more complex

13.2 Gaussian processes (uncertainty sampling, Bayesian optimization)

A GP puts a prior over functions with a kernel, e.g. RBF $k(x, x') = \exp\big(-\|x - x'\|^2/2\ell^2\big)$. Given training inputs X with targets $\mathbf{y}$ and noise variance σ_n^2:

Key formulas

$$\mu(x_*) = k(x_*, X)\big[K(X,X) + \sigma_n^2 I\big]^{-1}\mathbf{y}, \qquad \sigma^2(x_*) = k(x_*, x_*) - k(x_*, X)\big[K(X,X) + \sigma_n^2 I\big]^{-1}k(X, x_*).$$

- $\sigma^2(x_*)$ does not depend on $\mathbf{y}$, only on where we have sampled. Pure variance-based sampling therefore spreads points out (space filling): it "fills the gaps".
- The length scale ℓ sets how far one label informs its neighbours: small ℓ needs many samples, large ℓ assumes a smooth function.
- With high observation noise, uncertainty sampling can be no better than random sampling.
- For classifiers, margin or entropy sampling clusters labels along the decision boundary.

13.3 Semi-supervised learning: label propagation and spreading

Build a similarity graph over all points (labelled and unlabelled), usually with an RBF kernel, and let labels diffuse iteratively from labelled nodes to their neighbours, closer neighbours having more influence. Label spreading differs from propagation in its graph normalization (and allows labelled points to soften).

- Relies on the **cluster / manifold assumption**: nearby points (in the same dense region) share a label.
- Helps when classes form separated clusters: a handful of labels spreads to whole clusters.
- Hurts when classes overlap or a cluster bridges two classes (e.g. moons plus a Gaussian blob): wrong labels propagate across the boundary.
- Active + semi-supervised learning is not always better than either alone. Both can become overconfident when early labels are unlucky.

14 Code reference for the JupyterHub part

14.1 scikit-learn: linear models, CV, search

```
import numpy as np
from sklearn.linear_model import (LinearRegression, Ridge, Lasso, ElasticNet,
    HuberRegressor, SGDRegressor, LogisticRegression)
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import (cross_val_score, cross_validate, KFold,
    StratifiedKFold, GroupKFold, TimeSeriesSplit, GridSearchCV, RandomizedSearchCV,
    learning_curve, validation_curve)
from scipy.stats import loguniform, randint, uniform

# custom "linear" features, e.g. PS1 Model 4: a + b t + c t^2 + d sin(w t)
w = 2*np.pi / 52.0                      # weekly data, 1-year period
Phi = np.column_stack([t, t**2, np.sin(w*t)])
m = LinearRegression().fit(Phi, y);  m.intercept_, m.coef_

model = make_pipeline(PolynomialFeatures(15), StandardScaler(), Ridge(alpha=1e-3))
s = cross_val_score(model, X, y, cv=TimeSeriesSplit(n_splits=5),
                    scoring='neg_mean_squared_error')       # MSE = -s.mean()
cv = cross_validate(model, X, y, cv=GroupKFold(5), groups=g, return_train_score=True)

search = RandomizedSearchCV(model,
    {'ridge__alpha': loguniform(1e-8, 1e3), 'polynomialfeatures__degree': randint(1, 21)},
    n_iter=60, cv=5, scoring='neg_mean_squared_error', random_state=0).fit(X, y)
search.best_params_, -search.best_score_

sizes, tr, te = learning_curve(model, X, y, train_sizes=[50,100,200,500,1000,2000],
                               cv=TimeSeriesSplit(5), scoring='neg_mean_squared_error')
```

14.2 PCA and decompositions

```
from sklearn.decomposition import PCA, SparsePCA, DictionaryLearning, NMF
pca = PCA().fit(X)
evr = pca.explained_variance_ratio_                    # scree plot: plt.plot(evr, 'o-')
k95 = np.searchsorted(np.cumsum(evr), 0.95) + 1        # components for 95 %
Z = PCA(k95).fit_transform(X)                          # codes
```

```
X_rec = PCA(k95).fit(X).inverse_transform(Z)              # reconstruction

Xc = X - X.mean(0)                                       # by hand with SVD
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
evr_svd = S**2 / np.sum(S**2);  Z_svd = Xc @ Vt[:k].T     # = U[:, :k] * S[:k]
```

14.3 PyTorch autograd, Jacobians, finite-difference checks

```
import torch
x = torch.tensor([2.0, 5.0], requires_grad=True)
f = torch.log(x[0]) + x[0]*x[1] - torch.sin(x[1])
f.backward();  print(x.grad)                             # tensor([5.5000, 1.7163])

from torch.autograd.functional import jacobian
def F(x): return torch.stack([x[0]**2 + torch.sin(x[1]), x[0]*x[1] + torch.exp(x[0])])
J = jacobian(F, torch.tensor([2.0, torch.pi]))           # [[4,-1],[10.53,2]]

y = F(x); y[0].backward(retain_graph=True); g0 = x.grad.clone()
x.grad.zero_(); y[1].backward(); g1 = x.grad.clone()     # rows of J, one backward each

def fd(f, x0, h=1e-6):                                   # central differences
    g = np.zeros_like(x0)
    for i in range(len(x0)):
        e = np.zeros_like(x0); e[i] = h
        g[i] = (f(x0 + e) - f(x0 - e)) / (2*h)
    return g
torch.set_default_dtype(torch.float64)                   # when precision matters
```

14.4 Optimizing with autograd; constrained optimization with SciPy

```
p = torch.tensor([1.0, 1.0], requires_grad=True)
opt = torch.optim.Adam([p], lr=0.05)
for it in range(500):
    opt.zero_grad(); loss = loss_fn(p); loss.backward(); opt.step()

opt = torch.optim.LBFGS([p], lr=1, max_iter=20)
def closure():
    opt.zero_grad(); l = loss_fn(p); l.backward(); return l
for _ in range(5): opt.step(closure)

from scipy.optimize import minimize
cons = {'type': 'ineq', 'fun': lambda z: V(z) - 0.5}     # SciPy: fun(z) >= 0
res = minimize(D, x0=[2.5, 0.8], jac=grad_D, constraints=[cons],
               method='SLSQP', bounds=[(1e-3, None)]*2)  # jac=None -> 2-point FD
```

14.5 GAN training loop (correct version) and bugs to look for

```
import torch.nn as nn
G = nn.Sequential(nn.Linear(zd, h), nn.ReLU(), nn.Linear(h, h), nn.ReLU(), nn.Linear(h, xd))
D = nn.Sequential(nn.Linear(xd, h), nn.LeakyReLU(0.2), nn.Linear(h, h),
                  nn.LeakyReLU(0.2), nn.Linear(h, 1))     # outputs logits
# spectral norm: nn.utils.parametrizations.spectral_norm(nn.Linear(xd, h))
crit = nn.BCEWithLogitsLoss()
optG = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
optD = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))  # balanced!
for (real,) in loader:
    b = real.size(0); z = torch.randn(b, zd)
    fake = G(z)
    optD.zero_grad()                                     # --- discriminator
    lossD = crit(D(real), torch.ones(b, 1)) + crit(D(fake.detach()), torch.zeros(b, 1))
    lossD.backward(); optD.step()
    optG.zero_grad()                                     # --- generator
    lossG = crit(D(fake), torch.ones(b, 1))              # non-saturating
    lossG.backward(); optG.step()
```

Checklist for a "find the buggy line" question

- learning rates of G and D orders of magnitude apart (HS2025: `lr_g=1e-3`, `lr_d` about 10^{-6});
- missing `.detach()` on the fakes in the D step; G trained against the *fake* label 0;
- `BCELoss` on raw logits, or a sigmoid output combined with `BCEWithLogitsLoss`;

- missing `zero_grad()`; `optimizer.step()` of the wrong network;
- `torch.rand` (uniform) instead of `torch.randn`; noise dimension mismatch;
- Tanh/Sigmoid on G's output clipping the data range; D far larger than G or vice versa.

14.6 VAE, score matching, diffusion, flow matching

```
# VAE loss
mu, logvar = enc(x).chunk(2, dim=-1)
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)          # reparameterization
kl = 0.5*torch.sum(mu**2 + logvar.exp() - logvar - 1, dim=-1).mean()
loss = ((dec(z) - x)**2).sum(-1).mean() + beta*kl

# denoising score matching
eps = sigma*torch.randn_like(x)
loss_dsm = ((s(x + eps) + eps/sigma**2)**2).sum(-1).mean()

# DDPM (epsilon prediction)
t = torch.randint(0, T, (b,)); ab = alpha_bar[t][:, None]; e = torch.randn_like(x0)
loss_ddpm = ((eps_net(ab.sqrt()*x0 + (1-ab).sqrt()*e, t) - e)**2).mean()

# flow matching: training and Euler sampling
x0 = torch.randn_like(x1); t = torch.rand(b, 1); xt = (1-t)*x0 + t*x1
loss_fm = ((v(xt, t) - (x1 - x0))**2).mean()
x = torch.randn(n, d)
for k in range(N): x = x + v(x, torch.full((n, 1), k/N)) / N

# Langevin sampling with a learned score
x = torch.randn(n, d)
for k in range(K): x = x + a*s(x) + (2*a)**0.5 * torch.randn_like(x)
```

14.7 Metropolis–Hastings

```
w, lp = w0, log_post(w0); samples, acc = [], 0
for i in range(N):
    w_new = w + step*np.random.randn(*w.shape)
    lp_new = log_post(w_new)                    # log likelihood + log prior
    if np.log(np.random.rand()) < lp_new - lp:  # accept with prob min(1, ratio)
        w, lp, acc = w_new, lp_new, acc + 1
    samples.append(w)
samples = np.array(samples[burn_in:]); print('acceptance', acc/N)
```

15 Worked exam-style answers

15.1 HS2025 review problem 3: model diagnostics (3 points)

A client's 4-D sensor data, a GAN written by ChatGPT, the code and the diagnostic plots are given. Name the line causing the problem, explain why, and say which plot led you there.

Model answer

Line: the `train_gan(...)` call with `lr_g=1e-3` and `lr_d` around 10^{-6}: the generator learns about 1000 times faster than the discriminator. (The architectures themselves are reasonable: G is Linear/ReLU with 4 outputs, D is Linear/LeakyReLU with 1 output.)
Why: the discriminator barely learns, so it cannot tell real from fake and gives the generator an almost constant, uninformative signal. The generator then moves quickly to whatever few points currently fool the weak discriminator: **the generator overpowers the discriminator, causing mode collapse.**
Plots: $D(\text{real}) \approx D(\text{fake}) \approx 0.5$ and flat almost from the start; D loss constant (about 1.39); the sample-diversity curve is near zero with spikes; the pair plot shows the generated points piled onto one location or a thin sliver while the real data has several clusters in every dimension.

General recipe: (1) check the learning-rate ratio, labels, `detach` and loss function; (2) match the plots to the table in §8.2; (3) explain the mechanism in one sentence: who wins the minimax game, and what that does to the generator's gradient.

15.2 Review problem 2: matching noise levels in denoising score matching

Loss $J(\theta) = \mathbb{E}\left\|\mathbf{s}_\theta(\tilde{\mathbf{x}}) + \boldsymbol{\epsilon}/\sigma^2\right\|^2$ with $\sigma \in \{0.1, 0.5, 1.0\}$ on two concentric rings.

- Arrows well defined only on and very near the rings, messy or near-zero arrows and chaotic streamlines elsewhere: $\sigma = 0.1$.
- Arrows point towards the rings from a moderate distance and the two rings remain distinguishable: $\sigma = 0.5$.
- Smooth field everywhere, flowing almost radially to one central region, ring structure lost: $\sigma = 1.0$.

Justification: noise of scale σ sets where training samples exist (a band of width about σ around the data) and how strongly the learned density $p_\sigma = p * \mathcal{N}(0, \sigma^2 I)$ is blurred.

15.3 Review problem 1: gradient descent step sizes

See the worked example in §4: A ($\alpha = 0.025$) converges to the global minimum 3.326; B (0.10) reaches the right valley but bounces forever because $\alpha f'' > 2$; C (0.15) jumps to the left local minimum 0.393 (oscillating slightly); D (0.2) diverges.

15.4 Quick answers to likely conceptual questions

Question	Answer
Which CV for the CO_2 series and why?	Time-series split: a shuffled split trains on the future, giving an optimistic test MSE
Best model at small vs. large N (PS1)?	Small N: simplest model with the right structure (linear + sine, low variance). Large N: the most expressive correct model (quadratic + sine, low bias)
Why can the lowest training error give a bad test error?	Extra flexibility fits noise (e.g. an undetermined t^2 term extrapolating wildly): variance
Forward or reverse AD for a NN loss?	Reverse: millions of inputs, one output, so one backward pass gives the whole gradient
Forward or reverse AD for 2 parameters → 1000 trajectory states?	Forward: 2 passes instead of 1000
Is the backward pass slower than the forward pass?	Usually yes, by a small constant factor (stored graph, extra bookkeeping)
Why does AD give ≈ 0 sensitivity to h_0 for the bouncing ball?	h_0 affects the velocity only through the bounce timing (a discontinuous branch), which AD does not differentiate; finite differences reveal it
L1 or L2 when 40 candidate features and you want the few that matter?	L1 (LASSO); elastic net if the features are correlated
L1 or L2 for nearly redundant channels where stable coefficients matter?	L2 (ridge)
Why sample α or the learning rate log-uniformly?	Their effect is multiplicative across decades
Why does BO never re-evaluate an observed point?	$\sigma = 0$ there, so its UCB equals its value; an unexplored point always beats it
PCA vs. autoencoder?	PCA: linear, ordered, orthogonal, deterministic, closed form. AE: nonlinear, unordered, random initialization, needs tuning
Why spectral norm on a least-volume decoder?	Otherwise the network shrinks $\mathbf{z}$ and scales up the decoder weights to fake a small volume
Why do GANs fail when supports do not overlap?	JS is constant ($\log 2$), so there is no gradient; use Wasserstein / OT / WGAN-GP
Mode covering vs. mode seeking?	Forward $\mathrm{KL}(p\|q)$ covers (maximum likelihood); reverse $\mathrm{KL}(q\|p)$ seeks (VI)
VAE vs. flow vs. GAN?	Encoder needed: VAE or flow. Exact likelihood: flow. Fast sharp samples: GAN or latent model
Why the noise term in Langevin sampling?	Without it, samples converge to the nearest mode instead of sampling the distribution
Why predict $\boldsymbol{\epsilon}$ instead of the score in diffusion?	Same information ($\mathbf{s} = -\boldsymbol{\epsilon}/\sqrt{1-\bar{\alpha}_t}$) but better-scaled, more stable targets
Why does flow matching train faster than a CNF?	Simulation-free regression on $\mathbf{x}_1 - \mathbf{x}_0$: no ODE solve and no trace during training
Why is self-attention suited to point clouds?	Permutation equivariant; with pooling, invariant; points interact (unlike PointNet)
Why does VI underestimate uncertainty?	It minimizes reverse KL, which is mode seeking; mean-field ignores correlations
MH acceptance rate 99 %: good?	No: the step is too small, the chain mixes slowly
Why is REINFORCE noisy, and the fix?	It uses full Monte Carlo returns; subtract a baseline (advantage, actor–critic) and clip updates (PPO)
When does semi-supervised learning hurt?	Overlapping classes or bridging clusters: labels propagate across the boundary

15.5 Plot-reading checklist

- **Loss curves:** decreasing; flat (stuck, vanishing gradient, lr too small); oscillating (lr too high, minimax cycling); diverging ($\alpha > 2/\lambda$). A growing train/validation gap means overfitting.
- **GAN panel:** $D(\text{real}) \to 1$ and $D(\text{fake}) \to 0$: D wins. Both stuck at 0.5 from the start with low diversity: D useless, collapse. Oscillations: mode hopping.
- **Sample and pair plots:** count the covered modes; mass between modes (blurry VAE, forward-KL-like); everything on one mode (collapsed GAN, reverse-KL-like).
- **Latent interpolation:** smooth changes mean a good latent space; jumps or a constant output mean collapse.

- **Score and vector fields:** arrows should point towards high density; the noise level sets their reach and the blur.
- **Scree plot:** a sharp elbow means low linear intrinsic dimension; a slow decay means a nonlinear manifold or noise.
- **Validation and learning curves:** which side is bias and which is variance (§3).